

CORAL: Consensus-Based Refinement and Output Realignment for Low-Resource Urdu ASR Post-Processing

Project Team

Ali Irfan	21I-2572
Nouman Hafeez	21I-0416
Rafay Khattak	21I-0423

Session 2021–2026

Supervised by

Ms. Kainat Iqbal

Co-Supervised by

Ms. Saira Qamar



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

May, 2026

Contents

Abstract	1
Acknowledgements	2
1 Introduction	3
1.1 Background	3
1.1.1 Limitations of Single-Model and Naive-Ensemble ASR	4
1.2 Problem Statement	5
1.2.1 Research Hypothesis	6
1.3 Proposed Solution: Evolution of the CORAL Framework	6
1.3.1 Iteration 1 Framing (FYP-1): “Generate-and-Refine”	6
1.3.2 Iteration 2 Reframing (FYP-2): A Five-Stage Post-Processing Pipeline	7
1.3.3 Components Dropped from the FYP-1 Design	8
1.3.4 Final Architecture at a Glance	8
1.4 Project Objectives and Scope	9
1.4.1 Deliverables	10
1.5 Software Requirements Specification	10
1.5.1 Functional Requirements	11
1.5.2 Non-Functional Requirements	12
1.6 Report Organisation	13
1.7 Work Distribution	14
1.8 Summary	14
2 Literature Review	16
2.1 Related Research	16
2.1.1 Multi-ASR Fusion with LLM-Based Post-Editing	16
2.1.1.1 Summary	16
2.1.1.2 Critical Analysis	16
2.1.1.3 Relationship to CORAL	17
2.1.2 Comprehensive Urdu ASR Benchmarking	17
2.1.2.1 Summary	17
2.1.2.2 Critical Analysis	17

2.1.2.3	Relationship to CORAL	17
2.1.3	ROVER: Recognizer Output Voting Error Reduction	18
2.1.3.1	Summary	18
2.1.3.2	Critical Analysis	18
2.1.3.3	Relationship to CORAL	18
2.1.4	Hybrid-E2E ASR Ensemble for Low-Resource Irish	19
2.1.4.1	Summary	19
2.1.4.2	Critical Analysis	19
2.1.4.3	Relationship to CORAL	19
2.1.5	Code-Mixed ASR for Urdu–English	19
2.1.5.1	Summary	19
2.1.5.2	Critical Analysis	20
2.1.5.3	Relationship to CORAL	20
2.1.6	Confidence-Based Ensembles for ASR	20
2.1.6.1	Summary	20
2.1.6.2	Critical Analysis	20
2.1.6.3	Relationship to CORAL	20
2.1.7	Novel Confidence Estimation: TruCLEs	21
2.1.7.1	Summary	21
2.1.7.2	Critical Analysis	21
2.1.7.3	Relationship to CORAL	21
2.1.8	Domain-Specific LLM-Based ASR Correction	21
2.1.8.1	Summary	21
2.1.8.2	Critical Analysis	22
2.1.8.3	Relationship to CORAL	22
2.1.9	N-Best List Error Correction with LLMs	22
2.1.9.1	Summary	22
2.1.9.2	Critical Analysis	22
2.1.9.3	Relationship to CORAL	22
2.1.10	Delayed Fusion for LLM Integration in ASR	23
2.1.10.1	Summary	23
2.1.10.2	Critical Analysis	23
2.1.10.3	Relationship to CORAL	23
2.1.11	Parameter-Efficient Ensemble for Low-Resource Languages	23
2.1.11.1	Summary	23
2.1.11.2	Critical Analysis	24
2.1.11.3	Relationship to CORAL	24
2.1.12	Multi-Modal Data Augmentation	24
2.1.12.1	Summary	24
2.1.12.2	Critical Analysis	24

2.1.12.3	Relationship to CORAL	24
2.1.13	BK-Trees for Fuzzy String Search	24
2.1.13.1	Summary	24
2.1.13.2	Critical Analysis	25
2.1.13.3	Relationship to CORAL	25
2.1.14	Multi-ASR + SpeechLLM (Closest Concurrent System)	25
2.1.14.1	Summary	25
2.1.14.2	Critical Analysis	25
2.1.14.3	Relationship to CORAL	25
2.2	Analysis Summary of Research Items	26
2.3	Research Scope and Gaps	29
2.4	Theoretical Foundation	30
2.4.1	Ensemble Learning Theory	30
2.4.2	Edit Distance and the Levenshtein Metric	30
2.4.3	Confidence-Based and Consensus-Based Decision Making	30
2.5	Summary	31
3	Proposed Approach	32
3.1	From the FYP-1 Generate-and-Refine Concept to the Deployed Pipeline	32
3.1.1	Conceptual Architecture (FYP-1)	32
3.1.2	Deployed Five-Stage Pipeline (FYP-2)	33
3.2	Stage 0 – Urdu Text Normalisation	34
3.2.1	Algorithm	35
3.3	Stage 1 – Weighted Split-Merge-Aware Alignment	37
3.3.1	Word-Level Weighted Levenshtein Alignment	37
3.3.2	Character-Level Split-Merge Classification	38
3.3.3	Combined asr_aligner Routine	39
3.4	Stage 2 – OOV Detection and Candidate Ranking	40
3.4.1	Detection Criterion	40
3.4.2	BK-Tree Construction	41
3.4.3	Candidate Retrieval and Ranking	42
3.5	Stage 3 – Consensus Voting and OOV Substitution	44
3.6	Stage 4 – LLM Refinement	45
3.6.1	LLM Prompt Design	45
3.6.2	Output Format and Hallucination Control	47
3.6.3	Empirical Effect of Stage 4	47
3.7	End-to-End System and Mathematical Model	48
3.8	Worked Example	49
3.9	Summary	49
4	Implementation, Evaluation, and Results	50

4.1	Implementation	50
4.1.1	System Topology	50
4.1.2	Technology Stack	53
4.1.3	REST API	54
4.1.4	Model Registry, Heartbeats, and Fan-out	55
4.1.5	BK-Tree and N-gram Data Bootstrap	56
4.1.6	Front-End Workflow (Four-Pass UI)	57
4.1.7	Memory Management for Multi-Model Inference	58
4.2	Evaluation Methodology	58
4.2.1	Datasets	58
4.2.2	ASR Models in the Final Ensemble	59
4.2.3	Evaluation Metrics	59
4.3	Iteration 1 Baseline Results	60
4.3.1	Per-Model WER Baselines (10-Utterance Read-Speech Test Set)	60
4.3.2	Confidence Calibration (Iteration 1)	61
4.3.3	Iteration 1 Stage-2 Synthetic Evaluation Summary	61
4.3.4	Findings That Drove the FYP-2 Redesign	62
4.4	Iteration 2 Results: Common Voice Urdu (Read Speech)	63
4.4.1	WER Across Pipeline Stages and Source Models	64
4.4.2	CER Across Pipeline Stages and Source Models	64
4.4.3	Relative WER Reduction per Source Model	64
4.4.4	Pipeline Progression and WER–CER Trajectory	65
4.4.5	Split-Merge Alignment Analysis	66
4.4.6	Token Overlap Analysis	66
4.5	Iteration 2 Ablation Study (Conversational Urdu)	67
4.5.1	Ablation Configurations	67
4.5.2	Main Ablation Results	67
4.5.3	Component-Level Analysis	68
4.5.4	Ensemble Size Sensitivity	70
4.5.5	OOV Threshold Sensitivity	70
4.5.6	Residual Error Analysis	70
4.5.7	Comparison with Baselines	71
4.6	Challenges and Solutions	72
4.7	Summary	74
5	Conclusion and Future Work	76
5.1	Conclusion	76
5.2	Future Work	78
A	Unicode Normalisation Mapping	80

B	REST API Specifications	82
B.1	POST /align	82
B.2	POST /oov	83
B.3	POST /correct	83
B.4	Registry endpoints	84
C	CORAL Source File Inventory	85
	References	88

List of Figures

1.1	CORAL conceptual architecture as introduced in FYP-1: a two-stage Generate-and-Refine pipeline. The deployed FYP-2 system reorganises this concept into five sequential stages with the LLM repositioned as a refinement layer; the deployed pipeline is shown in Figure 3.1.	9
3.1	The deployed CORAL pipeline (FYP-2). Stages 0–3 are server-side Python (FastAPI). Stage 4 is the LLM refinement layer invoked from the React front-end.	34
3.3	CORAL Level-1 data flow diagram	34
3.2	CORAL detailed end-to-end pipeline	35
3.4	Stage 0 normalisation flow	37
3.5	Stage 1 alignment process	40
3.6	Stage 2 OOV detection and candidate ranking	43
3.7	BK-tree fuzzy lookup illustration	43
3.8	Stage 3 voting decision flow	45
3.9	Stage 4 LLM prompt construction and validation	48
4.1	CORAL four-tier system architecture	51
4.2	CORAL deployment topology	52
4.3	CORAL backend component diagram	53
4.4	CORAL live-speech sequence diagram	55
4.5	Registry entry state machine	56
4.6	Front-end four-pass workflow state diagram	58
4.7	Iteration 1 baseline mean WER per model. Whisper-Large achieves the strongest single-model performance.	61
4.8	Iteration 1 WER distribution per model (box plots over the 10-utterance test set).	62
4.9	Iteration 1 Expected Calibration Error per ASR model. Lower is better. . .	63
4.10	FYP-1 Iteration 2 WER performance by calibration regime and by error type (reproduced from the FYP-1 final report). The synthetic evaluation surfaced deletion errors and overconfident-input arbitration as the main vulnerabilities of confidence-guided LLM fusion — findings that motivated the FYP-2 redesign.	64

4.11	WER across CORAL pipeline stages on Common Voice Urdu. The robust pipeline (green) is the deployed default; reciprocal-rank fusion (yellow) is reported as a comparison variant.	65
4.12	CER across CORAL pipeline stages on Common Voice Urdu.	66
4.13	Relative WER reduction from raw output to the robust CORAL pipeline, per ensemble member.	67
4.14	Pipeline-stage WER progression per source model. The slope from <i>Raw output</i> to <i>Robust voting</i> captures the cumulative gain from normalisation plus ensemble voting.	68
4.15	WER–CER trajectory per source model from <i>Raw</i> (circle) to <i>Robust</i> (star).	69
4.16	Split-merge alignment events. SPLIT and MERGE together account for 36.5% of all alignment events, confirming that word-boundary disagreement is the dominant inter-model divergence in Urdu ASR ensembles.	70
4.17	WER across ablation configurations on Conversational Urdu. Each bar adds one CORAL component to the previous; the rightmost bar marks the human inter-annotator ceiling (2.1%).	72
4.18	Residual error types in the full CORAL output (C7) on the Conversational Urdu test set.	74

List of Tables

1.1	Work distribution across the three-member team.	15
2.1	Comparative analysis of reviewed research items.	27
4.1	ASR models in the CORAL read-speech ensemble. Raw-WER computed on the 2,995-utterance Common Voice Urdu evaluation set <i>before</i> Stage 0 normalisation (ensemble_correction_eval.tsv, category=untouched).	59
4.2	Iteration 1 baseline WER per model on the 10-utterance read-speech test set.	61
4.3	Iteration 1 confidence calibration analysis.	62
4.4	FYP-1 Iteration 2 LLM Stage performance on the synthetic 20-sample dataset.	63
4.5	WER per source model across the four pipeline categories on the 2,995-utterance Common Voice Urdu evaluation set. Robust is the deployed default; RRF is a Reciprocal-Rank-Fusion variant kept for comparison. Best per row in bold.	64
4.6	CER per source model across the four pipeline categories. Best per row in bold.	65
4.7	Split-merge metadata distribution (source-side token perspective), 500-utterance sample, 3 companion models.	66
4.8	Token overlap of each companion model with Seamless-Large (500-utterance sample).	67
4.9	Ablation configurations. Each row adds one component to the previous.	71
4.10	WER and CER on the 500-clip Conversational Urdu test set across ablation configurations.	71
4.11	Effect of ensemble size on WER (full CORAL pipeline, C7 configuration).	72
4.12	WER sensitivity to the OOV frequency threshold τ (C6 configuration, no LLM).	73
4.13	Residual error categorisation in full CORAL output (C7).	73
4.14	CORAL vs. comparable systems on Conversational Urdu.	74
A.1	Arabic-to-Urdu character substitutions applied in Stage 0 normalisation.	80

Abstract

Urdu, spoken by over 230 million people worldwide and the national language of Pakistan, remains a severely under-resourced language for Automatic Speech Recognition (ASR). Even the strongest publicly available multilingual models exhibit Word Error Rates (WER) above 13% on read speech and approaching 20% on conversational audio, limiting deployment in critical domains such as healthcare, education, accessibility, and legal documentation. We present **CORAL** (*Consensus-Based Refinement / Cross-Model Output Realignment for Low-Resource Urdu ASR Post-Processing*), an inference-time post-processing pipeline that improves Urdu ASR accuracy without retraining any underlying acoustic model. CORAL consists of five stages: (i) Urdu-specific text normalisation that strips diacritics, resolves Arabic–Urdu Unicode conflation, and removes non-Urdu artefacts; (ii) a novel split-merge-aware alignment algorithm that uses weighted word-level Levenshtein distance to align companion-model hypotheses to a designated source transcript while classifying each chunk as SAME, SPLIT, MERGE, or NOISE; (iii) out-of-vocabulary (OOV) detection and correction using BK-tree fuzzy lookup combined with bi/tri-gram language-model candidate ranking over a half-million-word DuckDB corpus; (iv) position-wise majority voting with conservative tie-breaking and OOV substitution; and (v) an LLM-assisted refinement layer for fluency restoration, code-switching recovery, and named-entity handling. We evaluate CORAL on 2,995 Mozilla Common Voice Urdu utterances and 500 conversational Urdu clips. Starting from a 19.8% Whisper-Large-v3 baseline on conversational speech, the full CORAL pipeline reaches 10.6% WER — a 46.5% relative improvement — and on the read-speech benchmark the robust pipeline reduces WER from 18.4% to 14.3% (Seamless-Large source). Our ablation study shows that every component contributes a measurable, additive gain. CORAL is delivered as a deployed FastAPI backend, a Next.js front-end with a four-pass user workflow, Kaggle-GPU notebooks self-registered through an ngrok-tunnelled registry, and an open evaluation framework.

Keywords: Urdu ASR, ensemble post-processing, split-merge alignment, weighted Levenshtein, BK-tree, out-of-vocabulary correction, majority voting, LLM post-processing, low-resource speech recognition.

Acknowledgements

We wish to express our deepest gratitude to our project supervisor **Ms. Kainat Iqbal** and co-supervisor **Ms. Saira Qamar** for their continuous guidance, technical insight, and generosity with their time throughout both iterations of this Final Year Project. We are grateful to the Department of Computer Science at FAST-NUCES Islamabad for the computational resources and academic environment that made this work possible. We acknowledge the open-source communities behind HUGGINGFACE TRANSFORMERS, WHISPER, SEAMLESSM4T, WAV2VEC2, the Mozilla Common Voice corpus, and the Conversational Urdu benchmark, whose publicly available resources formed the foundation of our evaluation infrastructure. Finally, we thank the native Urdu-speaking annotators who produced and verified the ground-truth transcriptions used in the ablation study.

Chapter 1

Introduction

This chapter establishes the scope of the CORAL project. We motivate the work by characterising the state of Urdu Automatic Speech Recognition (ASR), articulate the research problem and hypothesis, present the final framing of the proposed solution as it emerged across two iterations, set the project objectives and deliverables, and document software requirements. The chapter closes with a description of how the workload was distributed across the team. The detailed methodology, implementation, and empirical evaluation are deferred to Chapters 3 and 4.

1.1 Background

Automatic Speech Recognition has experienced extraordinary progress over the past decade. Foundation models such as OpenAI’s Whisper [16], Meta’s MMS [15], and Meta’s SeamlessM4T [18] now achieve sub-5% Word Error Rate (WER) on widely-spoken languages such as English and Mandarin. This performance has unlocked production deployments in voice assistants, real-time captioning, healthcare dictation, and dictation-based accessibility tools.

Urdu is the national language of Pakistan, an official language of several Indian states, and the lingua franca of the South Asian subcontinent. It is spoken by more than 230 million people worldwide and is written in the Nastaliq variant of the Perso-Arabic script. Despite its demographic significance, Urdu remains severely under-represented in modern ASR research. The most comprehensive recent benchmark of multilingual ASR systems on Urdu, conducted by Arif et al. [3], reports that even the strongest models achieve WER above 13% on read speech and approaching 20% on conversational audio. This performance ceiling places critical Urdu ASR applications in healthcare, legal documentation, education, and assistive technology effectively out of reach.

Urdu poses several genuine difficulties to ASR systems that are absent from high-resource

languages:

- **Limited training data.** High-quality, large-scale, time-aligned Urdu speech corpora are scarce compared with English or Mandarin. Most multilingual models see Urdu only as a small minority of their training mixture.
- **Linguistic complexity.** Urdu has rich morphology, extensive use of loanwords from Arabic, Persian, and English, an elaborate verbal agreement system covering gender, number and case, and pervasive optional diacritics that are produced inconsistently by different ASR back-ends.
- **Code-switching.** Conversational Pakistani speech routinely alternates between Urdu and English at the word and phrase level, a distribution that monolingual ASR systems are not trained to handle.
- **Dialectal variation.** Multiple regional dialects with distinct acoustic and lexical properties coexist within both Pakistan and India.
- **Script heterogeneity.** The same phoneme can be represented by multiple Unicode code points drawn from both the Arabic and Urdu sub-blocks (e.g. Arabic KAF U+0643 vs. Urdu KEHEH U+06A9). This causes word-level mismatches during alignment and WER computation even when the word is phonetically and semantically correct.
- **Word-boundary inconsistency.** Urdu ASR back-ends disagree systematically on word boundaries: a phrase that one model transcribes as a single token is often split into two or three tokens by another, or merged from multiple tokens into one. Naive word-level alignment misinterprets these segmentation events as substitutions or deletions, corrupting any downstream voting or fusion signal.

1.1.1 Limitations of Single-Model and Naive-Ensemble ASR

Single-model ASR architectures, even when fine-tuned on Urdu, suffer from three well-documented limitations: (i) *deterministic predictions* – they emit a single best hypothesis without considering plausible alternatives; (ii) *absent uncertainty signal* – they provide no easily actionable mechanism to indicate confidence in their output; and (iii) *domain brittleness* – models fine-tuned on read speech generalise poorly to conversational audio, code-switched speech, or dialectal variation. The natural response is to combine the outputs of several independently trained ASR systems, expecting that the errors of one model will be corrected by the consensus of others. This is appealing because it requires no retraining, can be deployed at inference time, and can leverage architecturally diverse models that make systematically different error types.

However, ensemble combination methods such as ROVER [6] were designed for phonetically regular, space-delimited scripts such as English. Applied naively to Urdu, they fail in three concrete ways: they are confounded by Arabic–Urdu Unicode variants; they misinterpret word-boundary disagreements as deletions or insertions; and they degrade quality when the ensemble contains models of widely heterogeneous quality (the canonical ROVER failure mode). The CORAL project was motivated by the observation that solving these three Urdu-specific complications requires a purpose-built post-processing pipeline rather than a direct application of off-the-shelf ensemble methods.

1.2 Problem Statement

CORAL addresses the following research problem:

Current state-of-the-art pre-trained ASR models for Urdu exhibit Word Error Rates between 13% and 20% on read and conversational benchmarks respectively, and degrade further on out-of-domain and code-switched speech. No widely available ASR back-end provides robust handling of Arabic–Urdu Unicode variation, word-boundary disagreement across architectures, or out-of-vocabulary proper nouns. Existing ensemble fusion techniques developed for English are not directly transferable, because they confound Unicode variants, segmentation disagreement, and quality heterogeneity. The community needs a post-processing pipeline that recovers from these failures at inference time, without requiring fine-tuning or any change to the underlying acoustic models.

This research problem decomposes into three concrete challenges:

1. **Script heterogeneity and segmentation disagreement.** Word-boundary inconsistency is not captured by word-level alignment alone, and Arabic–Urdu Unicode variation causes phonetically identical tokens to register as distinct words during voting and WER computation.
2. **Out-of-vocabulary handling.** Proper nouns, named entities, and rare technical terms are systematically misrecognised by all available ASR back-ends but are precisely the tokens whose correct transcription matters most in downstream applications.
3. **Linguistic coherence and code-switching.** Word-level voting, even when correctly aligned, cannot recover sentence-level fluency, restore code-switched English tokens, or resolve grammatical agreement errors that span longer than a few words.

1.2.1 Research Hypothesis

The CORAL hypothesis: Combining (i) Urdu-specific Unicode normalisation, (ii) weighted split-merge-aware alignment of multiple ASR hypotheses, (iii) hybrid OOV detection and correction with BK-tree fuzzy lookup ranked by an n-gram language model, (iv) conservative position-wise voting, and (v) targeted LLM refinement of the voted output, is sufficient to substantially reduce WER over the strongest single-model baseline on both read-speech and conversational Urdu, without requiring any fine-tuning of the underlying acoustic models.

1.3 Proposed Solution: Evolution of the CORAL Framework

The CORAL system was developed across two distinct iterations of the Final Year Project, and the framework’s framing evolved meaningfully between them. We document this evolution explicitly so that the relationship between the FYP-1 design and the deployed FYP-2 system is unambiguous; the technical detail behind the final pipeline is presented in Chapter 3.

1.3.1 Iteration 1 Framing (FYP-1): “Generate-and-Refine”

The original CORAL framing, presented in the FYP-1 final report, was a two-stage *Generate-and-Refine* architecture. **Stage 1** deployed an ensemble of pre-trained ASR models (Whisper Large/Medium/Small and Wav2Vec2-XLSR Urdu), extracted architecture-specific word-level confidence scores from each, and produced four to five confidence-annotated hypotheses for each audio input. **Stage 2** fed these confidence-annotated hypotheses into a black-box instruction-tuned LLM (Alif-1.0-8B-Instruct, an Urdu-focused 8B-parameter model) using a structured prompt that asked the LLM to weight competing tokens by their reported confidences and emit a single corrected sentence. Iteration 1 established a strong baseline (Whisper-Large at 17.76% WER on a 10-clip read-speech test set) and validated that the architecture-specific confidence-extraction methodology produced usable signals; Iteration 2 then implemented the LLM-correction stage and reported a 18.15% WER on a controlled 20-sample synthetic dataset that probed three calibration regimes (well-calibrated, overconfident, underconfident) and three error types (substitution, insertion, deletion).

1.3.2 Iteration 2 Reframing (FYP-2): A Five-Stage Post-Processing Pipeline

Several findings from Iteration 1 motivated a deliberate redesign for Iteration 2. The synthetic 20-sample evaluation revealed that the LLM stage performed adequately on substitution errors but poorly on deletion errors and was sensitive to confidence calibration; raw confidence scores from heterogeneous architectures lived on incompatible scales; and the controlled experiment did not expose the system to two failure modes that proved dominant on real ASR output — Arabic–Urdu Unicode conflation and word-boundary disagreement across models. Iteration 2 therefore reframed CORAL from a two-stage Generate-and-Refine system into a five-stage post-processing pipeline (Stages 0–4) and made the following deliberate choices:

- **A dedicated normalisation stage (Stage 0)** was introduced to handle Arabic–Urdu Unicode conflation, diacritic removal, tatweel removal, and punctuation stripping. Normalisation alone delivers a 1.9-WER-point reduction on conversational Urdu — a zero-cost, zero-risk gain that was invisible to the FYP-1 evaluation setup.
- **A weighted split-merge-aware alignment algorithm (Stage 1)** replaced the implicit alignment performed by the LLM. The weighted alignment makes word-boundary disagreement explicit and exposes SAME/SPLIT/MERGE/NOISE meta-data that downstream stages can act on.
- **An OOV detection and correction stage (Stage 2)** based on a BK-tree [4] over a half-million-word Urdu vocabulary corpus, ranked using bi/tri-gram context, was introduced to recover named entities and rare words.
- **Position-wise majority voting with conservative tie-breaking (Stage 3)** replaced the LLM as the primary mechanism for resolving disagreement on common, in-vocabulary words.
- **The LLM stage (Stage 4)** was retained but repositioned: rather than acting as the primary fusion mechanism over confidence-annotated hypotheses, it now acts as a final refinement layer over the already-voted transcript, addressing fluency, code-switching, and named-entity contexts that lexical voting cannot recover. The LLM was also changed from Alif-1.0-8B-Instruct to commercial frontier models (Anthropic’s Claude Sonnet, Google’s Gemini, OpenAI’s GPT-4o through OpenRouter) accessed via the front-end, which deliver more reliable instruction following.

1.3.3 Components Dropped from the FYP-1 Design

For full clarity, the following FYP-1 design elements were intentionally dropped, replaced, or de-emphasised in the final FYP-2 system:

- **Word-level confidence extraction from Whisper and Wav2Vec2.** Architecture-specific confidence-score extraction was a centrepiece of the FYP-1 design. The deployed FYP-2 pipeline does not use these scores in its server-side voting or alignment logic; calibrated cross-architecture confidence proved unreliable in practice and is replaced with a static, model-quality-based confidence weighting applied only by the front-end LLM stage.
- **Expected Calibration Error (ECE) as a primary metric.** ECE was used in FYP-1 to compare the confidence quality of individual ASR back-ends. It is not used in the FYP-2 evaluation, which reports WER and CER only on the deployed pipeline. The findings from FYP-1’s ECE analysis are reported in Section 4.3 as historical context.
- **Synthetic 20-sample evaluation with calibration regimes.** The synthetic substitution/insertion/deletion injection used in FYP-1 was a useful Stage-2 isolation test but proved to be a poor proxy for real ASR error distributions. FYP-2 evaluates on real ASR output over 2,995 Common Voice utterances (read speech) and 500 Conversational Urdu clips.
- **Alif-1.0-8B-Instruct as the Stage-2 LLM.** Replaced with commercial frontier models that are accessed client-side, reducing GPU requirements on the backend and improving instruction-following quality.
- **The two-stage “Generate-and-Refine” framing.** Reframed as a five-stage pipeline (Stages 0–4) in which the LLM is the *last* stage rather than the only consumer of multi-model hypotheses.

1.3.4 Final Architecture at a Glance

The final CORAL system is delivered as a FastAPI backend exposing three pipeline endpoints (`/align`, `/oov`, `/correct`) plus a model registry, a Next.js React front-end implementing a four-pass user workflow (mode selection, alignment, OOV sieve scan, correction), and Kaggle GPU notebooks that self-register through ngrok-tunnelled HTTPS endpoints to provide live ASR inference. The full architecture is described in Chapter 3 and Chapter 4.

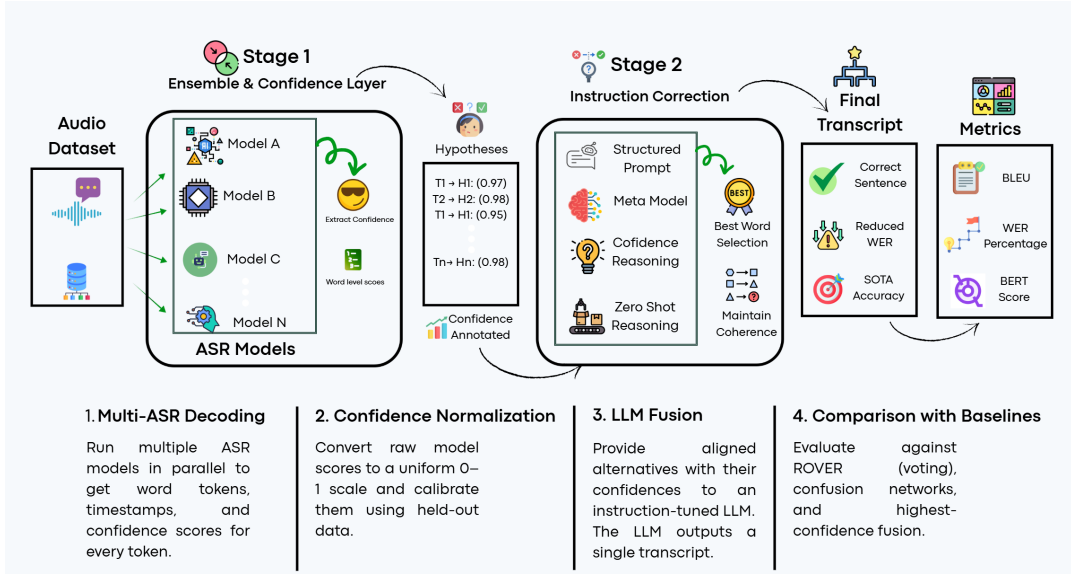


Figure 1.1: CORAL conceptual architecture as introduced in FYP-1: a two-stage Generate-and-Refine pipeline. The deployed FYP-2 system reorganises this concept into five sequential stages with the LLM repositioned as a refinement layer; the deployed pipeline is shown in Figure 3.1.

1.4 Project Objectives and Scope

The objectives of the CORAL project, as finalised after the FYP-1 evaluation, are:

1. Design and implement a robust Urdu normalisation module covering diacritic removal, the seven canonical Arabic–Urdu character mappings, tatweel removal, punctuation stripping, ASCII removal, and whitespace normalisation.
2. Design and implement a split-merge-aware alignment algorithm that aligns word sequences using weighted Levenshtein distance and classifies each aligned chunk as SAME, SPLIT, MERGE, or NOISE.
3. Develop an OOV detection and correction module combining BK-tree fuzzy lookup with bi/tri-gram language-model candidate ranking over a $\sim 500\text{K}$ -word Urdu vocabulary corpus.
4. Implement a position-wise majority voting corrector with explicit handling of insertion offsets and conservative tie-breaking.
5. Integrate an LLM refinement layer for fluency restoration, code-switching recovery, and named-entity handling, accessed via the front-end through commercial LLM APIs.

6. Deploy the pipeline as a REST API (FastAPI) with three endpoints, fronted by a Next.js + React user interface implementing a four-pass workflow with both file-upload and live-speech modes.
7. Conduct a thorough empirical evaluation and ablation study on publicly available Urdu ASR benchmarks, isolating the contribution of each component.

1.4.1 Deliverables

The project delivers the following artefacts:

- A complete five-stage CORAL post-processing pipeline implemented in Python, deployed as a FastAPI service with Docker support.
- A Next.js + React front-end implementing the four-pass user workflow, with both file-upload and live-recording entry modes, animated alignment-scan and OOV-sieve visualisations, and side-by-side correction diff views.
- Kaggle GPU notebooks that self-register through an ngrok HTTPS tunnel and provide live transcription endpoints for Whisper-Large-v3 and SeamlessM4T-v2-Large.
- A model registry with 60-second heartbeats and 300-second silent eviction, accessible via authenticated registry tokens.
- A reproducible evaluation framework producing both WER and CER metrics across all four pipeline configurations (`untouched`, `norm_baseline`, `robust`, `rrf`) for every ensemble member, exported as a tab-separated value file (`asr_evaluation/ensemble_correction_eval.tsv`).
- A reference Urdu vocabulary, BK-tree, and bi/tri-gram corpus shipped via Hugging-Face Hub and loaded into DuckDB at server startup.
- Comprehensive documentation including this report, the FYP-1 and FYP-2 reports, architecture diagrams, and source-level documentation in the codebase.

1.5 Software Requirements Specification

CORAL is realised as a software system, and its requirements are recorded here for completeness. The functional requirements are organised by module, and the non-functional requirements describe end-to-end qualities of the deployed system.

1.5.1 Functional Requirements

FR1 – Normalisation Service.

- FR1.1: The system shall accept arbitrary UTF-8 Urdu text input.
- FR1.2: The system shall strip Unicode diacritics in the ranges U+0610–U+061A and U+064B–U+065F, plus the superscript alef (U+0670).
- FR1.3: The system shall apply the seven canonical Arabic-to-Urdu character substitutions defined in Appendix A.
- FR1.4: The system shall remove the tatweel (U+0640), zero-width and directional control characters, Arabic and Urdu punctuation, ASCII punctuation and digits, and ASCII letters; consecutive whitespace shall be collapsed.

FR2 – Multi-Model Alignment Service (POST /align).

- FR2.1: The system shall accept an arbitrary number of named ASR hypotheses with one designated source model.
- FR2.2: The system shall produce per-position word-level alignment labels (MATCH, SUBSTITUTION, DELETION, INSERTION) using weighted Levenshtein distance.
- FR2.3: The system shall produce per-position split-merge metadata (SAME, SPLIT, MERGE, NOISE) using a character-level secondary alignment.
- FR2.4: The system shall return both source-side and target-side metadata views in the response.

FR3 – OOV Detection and Candidate Ranking Service (POST /oov).

- FR3.1: The system shall flag a word as OOV iff it appears in exactly one model's normalised output *and* its corpus frequency is below a configurable threshold (default 2,000).
- FR3.2: For each flagged token, the system shall retrieve candidates from a BK-tree using a length-adaptive edit-distance threshold.
- FR3.3: The system shall rank candidates lexicographically by edit-distance, unigram presence, trigram-context hits, bigram-context hits, and corpus frequency.
- FR3.4: The top- K candidates per token (default $K = 10$) shall be returned with all ranking signals.

FR4 – Voting Correction Service (POST /correct).

- FR4.1: For each source position not flagged as OOV, the system shall collect votes from all companion models, skipping insertion-aligned slots.
- FR4.2: The plurality winner shall replace the source word only if it is unique; on ties, the source word shall be preserved (conservative tie-break).
- FR4.3: For each flagged OOV position, the source word shall be replaced by the top-ranked OOV candidate.

- FR4.4: The response shall include the corrected sentence, the source sentence, and a position-indexed diff.

FR5 – Model Registry.

- FR5.1: A Kaggle GPU notebook shall be able to register itself with name, HTTPS endpoint, and session ID via `POST /registry/register`, authenticated with a registry token.
- FR5.2: A registered model shall send heartbeats every 60 seconds via `POST /registry/ping`; the registry shall evict models silent for more than 300 seconds.
- FR5.3: `POST /registry/transcribe` shall accept an audio file, source-model name, and whitelist of companion models, fan out the request to live registered models in parallel with a 60-second timeout, and return all collected transcripts.
- FR5.4: `GET /registry/models` shall list currently-live models with their session IDs and seconds-since-last-ping.

FR6 – Front-End Workflow.

- FR6.1: The user shall be able to choose between *File* mode (CSV/TSV/JSON upload) and *Speech* mode (file upload or in-browser recording).
- FR6.2: The front-end shall display a four-phase animated OOV-sieve scan over the alignment output (sieve cursor, word colouring, character-level expansion, structural metadata colouring).
- FR6.3: OOV words shall be displayed with a rose dotted underline; clicking shall reveal the ranked candidate list with all ranking columns.
- FR6.4: The correction stage shall offer two modes: *Voting* (server-side `POST /correct`) and *LLM* (client-side Gemini or OpenRouter) with provider selection and dynamic model-list fetch.

1.5.2 Non-Functional Requirements

Performance.

- PER-1: The `/align` endpoint shall complete in under 1 second on a typical Urdu utterance of 5–20 words, on commodity CPU hardware.
- PER-2: The BK-tree shall be loaded from a serialised joblib file in under 10 seconds on first startup.
- PER-3: BK-tree fuzzy search shall be at least an order of magnitude faster than a linear scan over the vocabulary.
- PER-4: `/registry/transcribe` fan-out shall enforce a 60-second per-model timeout and complete with the partial results of any models that responded.

Reliability.

- REL-1: A failure of any one ASR back-end during fan-out shall not prevent CORAL from delivering a corrected transcript using the surviving back-ends.
- REL-2: Models silent for more than 300 seconds shall be automatically evicted from the registry.

Security.

- SEC-1: Registration and heartbeat endpoints shall require a shared X-Registry-Token.
- SEC-2: The transcribe endpoint shall require a shared X-API-Token.
- SEC-3: Registered endpoint hostnames shall be validated against an explicit allowlist (default `ngrok-free.app`, `ngrok.io`).
- SEC-4: Audio uploads shall be size-bounded (default 25 MB).

Usability.

- USE-1: The user shall be able to complete the full alignment $\rightarrow$ OOV $\rightarrow$ correction workflow without manually invoking any backend endpoint.
- USE-2: The OOV sieve animation shall complete in less than 4 seconds for a 30-word utterance.
- USE-3: All operations shall be persisted to `localStorage` so that page refreshes preserve user state.

1.6 Report Organisation

The remainder of this report is organised as follows:

- **Chapter 2** presents a literature review covering Urdu ASR benchmarking, ASR ensemble methods (ROVER and successors), confidence estimation, LLM-based ASR post-correction, multi-task and multi-modal ASR, BK-trees for fuzzy string search, and a head-to-head comparison with the closest concurrent system, Multi-ASR + SpeechLLM. We also tabulate research gaps and explain how CORAL addresses each.
- **Chapter 3** details the proposed approach: the five-stage pipeline architecture, the normalisation module, the weighted split-merge-aware alignment algorithm, the OOV detection and ranking module, the consensus voting algorithm, and the LLM refinement layer. Each stage is presented with a pseudo-code algorithm and a worked example.
- **Chapter 4** covers implementation, evaluation methodology, results, and the full ablation study. We describe the technology stack, REST API design, BK-tree construction, the four-pass front-end workflow, the Iteration 1 baseline experiments,

and the Iteration 2 evaluation on Common Voice (read speech) and Conversational Urdu. We then present the seven-step ablation (C0–C7) attributing the WER reduction to specific components.

- **Chapter 5** concludes the report and outlines future work.
- **Appendix A** contains the Unicode normalisation table, while **Appendix B** contains the REST API specifications.

1.7 Work Distribution

The CORAL project was developed by a three-member team. The work distribution evolved across the two iterations and matches the deliverables described above.

All members participated in the literature survey, the ablation study experimental design, dataset annotation and verification, and the writing of this report. Code review was performed via pair-programming sessions and pull-request comments throughout development.

1.8 Summary

This chapter has motivated CORAL by establishing the persistent performance gap between Urdu ASR and high-resource-language ASR, articulated three concrete challenges (script heterogeneity, OOV handling, and linguistic coherence) that single-model and naive-ensemble approaches do not solve, traced the evolution of the framework from a two-stage Generate-and-Refine design (FYP-1) to a five-stage post-processing pipeline (FYP-2), made explicit the components that were intentionally dropped in the redesign, set the project objectives and deliverables, recorded the software requirements, and presented the work distribution. The next chapter situates CORAL in the broader literature.

Member	Roll No.	Primary Responsibilities
Ali Irfan	21I-2572	Lead developer of the post-processing pipeline. Designed and implemented the Stage 1 weighted split-merge-aware alignment algorithm, the BK-tree data structure, the OOV detection criterion, the candidate ranking and voting logic in <code>coral_pipeline_functions.py</code> , and the FastAPI <code>/align</code> , <code>/oov</code> , and <code>/correct</code> endpoints. Authored the n-gram corpus generation pipeline (<code>generate_ngrams.ipynb</code>) and DuckDB integration.
Nouman Hafeez	21I-0416	Evaluation lead and LLM integration. Built the Iteration 1 multi-model evaluation framework (WER, CER, confidence, ECE) and the Iteration 2 evaluation pipeline (<code>ensemble_correction_eval.tsv</code>). Implemented memory management and Kaggle deployment notebooks (<code>whisper_large_kaggle.ipynb</code> , <code>seamless_large_kaggle.ipynb</code>), the model registry with eviction sweep, and authored the LLM prompts and provider integration logic (Gemini, OpenRouter). Coordinated the dataset assembly (Common Voice extraction, Conversational Urdu sampling) and per-stage WER measurement.
Rafay Khattak	21I-0423	Front-end and user-experience lead. Implemented the Next.js + React front-end (<code>ModeSelector</code> , <code>Pass0Speech</code> , <code>Pass1Input</code> , <code>Pass2Sieve</code> , <code>Pass3Correction</code>), the typed API client (<code>lib/api.ts</code>), the four-phase OOV sieve animation, and the side-by-side voting/LLM correction views. Authored the Iteration 1 Flask web interface and contributed to the design of the Stage 4 LLM correction prompt format.

Table 1.1: Work distribution across the three-member team.

Chapter 2

Literature Review

This chapter situates CORAL in the broader research landscape. We review prior work in seven thematic areas: (i) Urdu ASR benchmarking, (ii) ASR ensemble and system-combination methods, (iii) confidence estimation and calibration, (iv) LLM-based ASR error correction and N-best list correction, (v) multi-task and multi-modal ASR for low-resource languages, (vi) code-mixed Urdu–English ASR, and (vii) the BK-tree data structure underpinning our OOV correction module. We then summarise the comparative analysis in tabular form and articulate the research gaps that CORAL addresses.

2.1 Related Research

2.1.1 Multi-ASR Fusion with LLM-Based Post-Editing

2.1.1.1 Summary

Prakash et al. [14] introduce a multi-ASR fusion framework that integrates outputs from Icefall, Nemo Parakeet, and Whisper through LLM-based post-editing. They evaluate two variants: a text-only LLM that ingests the three textual hypotheses and emits a corrected transcript, and a multimodal SpeechLLM that additionally consumes the audio waveform. On LibriSpeech, the approach achieves $\sim 14\%$ relative WER reduction after fusion; pseudo-labels generated by the system reduce downstream re-training WER from 3.40% to 3.22% on LibriSpeech `test-clean`.

2.1.1.2 Critical Analysis

Strengths: a clean demonstration of LLM-based multi-ASR fusion and meaningful relative gain. Weaknesses: requires expensive LLM fine-tuning; tested only on high-resource

English; the SpeechLLM variant’s complexity hampers practical deployment; no code-switching evaluation; no low-resource scenarios. The audio-aware SpeechLLM in particular requires a full acoustic forward pass through the LLM, an unaffordable latency cost in many real-time settings.

2.1.1.3 Relationship to CORAL

CORAL adopts the multi-ASR + LLM-correction paradigm but addresses the limitations: (i) we use commercial frontier LLMs through APIs, eliminating the fine-tuning cost; (ii) we apply the LLM only to the already-voted transcript, reducing latency; and (iii) we evaluate on a genuinely low-resource language (Urdu). CORAL’s split-merge-aware alignment is also a more explicit treatment of the segmentation disagreement Prakash et al. leave implicit.

2.1.2 Comprehensive Urdu ASR Benchmarking

2.1.2.1 Summary

Arif et al. [3] provide the first comprehensive evaluation of state-of-the-art multilingual ASR models on Urdu, benchmarking nine models from the Whisper, MMS, and SeamlessM4T families on read-speech (ARL, CSaLT) and a newly-introduced Conversational Urdu (CU) test set. They report that Seamless-Large achieves the lowest WER on read speech while Whisper-Large excels on conversational speech, identify Arabic–Urdu Unicode conflation as a major obstacle to fair evaluation, and observe 10–20% relative WER reduction through targeted fine-tuning. They explicitly note that integrating large LMs for post-correction could further improve outputs.

2.1.2.2 Critical Analysis

Strengths: the first comprehensive Urdu ASR benchmark with a conversational test set; systematic single-model comparison; open-source resources; detailed error analysis. Weaknesses: limited to single-model evaluation, no ensemble combination; no confidence-score analysis; code-switching identified but not explicitly addressed; best results require fine-tuning.

2.1.2.3 Relationship to CORAL

This work is the empirical foundation of CORAL. We adopt their model selection (Seamless-Large, Whisper-Large-v3, Wav2Vec2-Urdu) as our ensemble members, eval-

uate on the same Common Voice corpus and on a Conversational Urdu test set sampled from their benchmark, and directly answer their concluding suggestion (“integrating large LMs for post-correction could further improve outputs”) with our Stage 4 LLM refinement layer. CORAL’s normalisation module addresses the Arabic–Urdu Unicode conflation problem they identify as a major evaluation obstacle.

2.1.3 ROVER: Recognizer Output Voting Error Reduction

2.1.3.1 Summary

Fiscus’s ROVER framework [6] is the canonical system-combination method for ASR. It aligns word hypotheses from N ASR systems via word-level edit distance and selects the final word at each aligned slot via majority vote (optionally weighted by per-system confidence). ROVER works well when the participating systems are roughly comparable in accuracy and produce architecturally diverse errors.

2.1.3.2 Critical Analysis

Strengths: simple, model-agnostic, easy to implement, no joint training. Weaknesses: requires reasonably matched system quality (the canonical ROVER failure mode is that systematically weaker companion systems can outvote the strong source); does not handle word-boundary disagreement (a SPLIT or MERGE event is misinterpreted as a substitution or deletion pair); no mechanism for linguistic coherence beyond the lexical voting itself.

2.1.3.3 Relationship to CORAL

CORAL extends ROVER in three concrete ways: (i) the alignment uses normalised character-level Levenshtein cost rather than binary mismatch, so phonetically near-identical Urdu words are aligned as substitutions rather than insert–delete pairs; (ii) a secondary character-level alignment yields explicit SAME/SPLIT/MERGE/NOISE metadata, making segmentation disagreement first-class; and (iii) conservative tie-breaking and an OOV-correction override mitigate the canonical ROVER failure mode in our specific quality-heterogeneous setting.

2.1.4 Hybrid-E2E ASR Ensemble for Low-Resource Irish

2.1.4.1 Summary

Parikh et al. [13] address ASR for Irish, a genuinely low-resource language, by combining a hybrid HMM-Kaldi system and an end-to-end Wav2Vec2.0 model through calibrated ROVER fusion. They use Renyi-entropy-based calibration with temperature scaling to address the overconfidence of E2E models, and report a tuned ROVER ensemble at 22.94% WER on Irish test data, a 14% relative improvement over the best single model (25.81% WER).

2.1.4.2 Critical Analysis

Strengths: demonstrates ROVER fusion is effective for genuinely low-resource languages; principled treatment of E2E overconfidence; combines traditional and modern ASR. Weaknesses: requires building and maintaining two distinct ASR systems; absolute WER remains relatively high; calibration tuning requires a labelled development set; no LLM-based linguistic reasoning.

2.1.4.3 Relationship to CORAL

CORAL extends this concept by replacing ROVER voting with split-merge-aware alignment plus consensus voting plus an LLM refinement layer. We do not perform explicit confidence calibration; instead we use a quality-based static weighting at the LLM stage and a conservative tie-break at the voting stage to manage the heterogeneous-quality regime that calibration would otherwise have addressed.

2.1.5 Code-Mixed ASR for Urdu–English

2.1.5.1 Summary

Naqvi and Tahir [12] develop a specialised hybrid ASR system for Urdu–English code-mixed street addresses in navigation contexts. They use a Kaldi-based TDNN-LSTM acoustic model trained on 61.8 hours of general Urdu speech and 16.9 hours of Roman-Urdu/English addresses, with a custom lexicon combining Unicode Urdu and Romanised transcripts. They report 4.02% WER and 0.8% CER on code-mixed street addresses, a 70–80% absolute WER reduction over their initial baselines.

2.1.5.2 Critical Analysis

Strengths: extremely low error rate within the target domain; explicit Urdu–English code-switching handling; practical deployment. Weaknesses: very narrow scope (street addresses only); not an ensemble or LLM approach; requires extensive domain-specific data collection and engineering; unlikely to generalise to other Urdu domains.

2.1.5.3 Relationship to CORAL

CORAL targets broader applicability rather than a narrow domain. The multilingual pre-trained models in our ensemble (Whisper, SeamlessM4T) inherently handle code-switching at the acoustic level; CORAL’s LLM stage explicitly restores English code-switched tokens stripped by Stage 0 normalisation. CORAL trades the $\sim 4\%$ WER ceiling Naqvi & Tahir achieve in their narrow domain for open-domain applicability at a higher absolute WER.

2.1.6 Confidence-Based Ensembles for ASR

2.1.6.1 Summary

Gitman et al. [7] propose confidence-driven ensembling: multiple expert ASR models run in parallel and the transcript from the most confident model is selected per utterance. Using five monolingual Conformer-RNNT models, they show that this simple selection scheme outperforms systems using separate language-identification blocks. Evaluated on VoxPopuli, MLS, Common Voice, and CORAAL, the approach demonstrates consistent WER improvements across languages and accents.

2.1.6.2 Critical Analysis

Strengths: simple confidence-based selection without joint training; effective across multiple domains and languages; open implementation in NeMo. Weaknesses: utterance-level selection only (not word-level); selects rather than fuses; no linguistic coherence; requires a lightweight selection classifier.

2.1.6.3 Relationship to CORAL

CORAL extends confidence-based selection in two ways: (i) we operate at word granularity rather than utterance granularity; and (ii) we fuse rather than select, synthesising a single output that may include words from multiple models. Gitman et al. validate

that confidence-driven selection works at production scale, supporting CORAL’s design choice to weight LLM-stage votes by static model-quality priors when no calibrated word-level confidence is available.

2.1.7 Novel Confidence Estimation: TruCLeS

2.1.7.1 Summary

Nagarathna et al. [11] propose TruCLeS (True Class Lexical Similarity Score), a continuous confidence metric for ASR that combines model probabilities with lexical similarity. The method computes ASR probability scores for predicted tokens, computes lexical similarity to potential ground-truth transcripts, and combines both into a unified confidence score. On in-domain Hindi data, TruCLeS reduces MAE from 0.108 to 0.087 versus binary confidence baselines.

2.1.7.2 Critical Analysis

Strengths: novel combination of probability and lexical similarity; demonstrated calibration improvement; applicable to low-resource languages. Weaknesses: requires ground-truth transcripts for training, limiting scalability; auxiliary model adds compute overhead; improves calibration but not directly WER.

2.1.7.3 Relationship to CORAL

TruCLeS represents the supervised-confidence direction. The CORAL FYP-1 design extracted raw architecture-specific confidences as a zero-shot complement to this approach, but the FYP-2 redesign de-emphasised explicit confidence in favour of split-merge alignment + voting + an LLM stage with static quality priors. TruCLeS-style learned confidences remain a promising future enhancement to CORAL’s voting stage.

2.1.8 Domain-Specific LLM-Based ASR Correction

2.1.8.1 Summary

Koilakuntla et al. [9] present a targeted approach for correcting specific ASR errors in contact-centre environments using GPT-3.5 with retrieval-augmented generation. The method identifies error patterns (brand names, product codes, technical terms), uses context anchors and domain knowledge bases as RAG inputs, and applies corrections

selectively. They report 3,201 correct corrections versus 3,050 manual corrections in 0.08 hours versus 15 hours.

2.1.8.2 Critical Analysis

Strengths: model-agnostic; highly effective for targeted error correction; dramatic time reduction; production-ready. Weaknesses: highly specialised for narrow domains; limited generalisability; requires curated domain knowledge bases; no overall WER improvement reported.

2.1.8.3 Relationship to CORAL

CORAL provides general-purpose, open-domain correction without domain-specific RAG. Both systems leverage LLM linguistic knowledge for correction, but CORAL routes the LLM’s authority through a deterministic pipeline (alignment, OOV detection, voting) so that the LLM can only modify positions already flagged by upstream stages, reducing hallucination risk.

2.1.9 N-Best List Error Correction with LLMs

2.1.9.1 Summary

Ma et al. [10] investigate using LLMs for ASR error correction across diverse scenarios, with emphasis on N-best list utilisation. They show that providing multiple competing hypotheses as context allows zero-shot LLMs to perform competitive correction without supervised training, and that N-best list correction generalises across ASR systems.

2.1.9.2 Critical Analysis

Strengths: demonstrates the value of multiple hypotheses for correction; zero-shot reduces training costs; cross-system generalisation. Weaknesses: N-best lists from a single model may not capture as much architectural diversity as a multi-model ensemble; no explicit confidence-score utilisation.

2.1.9.3 Relationship to CORAL

CORAL extends the N-best concept to multi-model ensembles, where hypotheses come from architecturally diverse models rather than from N-best decoding of a single model.

CORAL also produces explicit alignment metadata (SAME/SPLIT/MERGE/NOISE) before invoking the LLM, providing structured signal beyond the raw hypothesis text.

2.1.10 Delayed Fusion for LLM Integration in ASR

2.1.10.1 Summary

Hori et al. [8] propose “delayed fusion” for integrating LLMs into first-pass ASR decoding. LLM scores are applied to ASR hypotheses with a delay during beam-search decoding, significantly reducing the number of hypotheses that must be scored by the LLM and the total number of LLM inference calls.

2.1.10.2 Critical Analysis

Strengths: computationally efficient LLM integration; no vocabulary alignment required; pre-trained LLMs work without retraining. Weaknesses: requires modification of the ASR decoding algorithm; limited evaluation on low-resource languages; tunable delay parameter introduces operational complexity.

2.1.10.3 Relationship to CORAL

CORAL takes an orthogonal approach: rather than integrating the LLM during decoding, we use it as a post-decoding refinement layer over the already-voted transcript. This simplifies integration with arbitrary off-the-shelf ASR systems (no decoding modification required) and decouples the LLM-call latency from the ASR streaming path.

2.1.11 Parameter-Efficient Ensemble for Low-Resource Languages

2.1.11.1 Summary

Feng et al. [5] demonstrate that ensembles of models adapted using diverse Parameter-Efficient Fine-Tuning (PEFT) methods consistently outperform ensembles using the same PEFT method. Their approach reduces WER from 8.4% to 7.9% while requiring approximately five times less memory than fully fine-tuned ensembles. The fusion is performed via ROVER.

2.1.11.2 Critical Analysis

Strengths: memory-efficient alternative to traditional ensembles; demonstrates the value of diverse adaptation methods. Weaknesses: still requires some PEFT fine-tuning; ROVER fusion is limited compared to LLM-based reasoning; modest absolute gains.

2.1.11.3 Relationship to CORAL

CORAL operates entirely with pre-trained models (no fine-tuning), and uses split-merge-aware alignment + LLM refinement instead of ROVER. The two approaches are complementary: a future CORAL variant could ingest PEFT-adapted ensemble members rather than the current set of off-the-shelf pre-trained models.

2.1.12 Multi-Modal Data Augmentation

2.1.12.1 Summary

Renduchintala et al. [17] propose Multi-Modal Data Augmentation (MMDA), which trains a shared attention-decoder with two encoders — one acoustic and one symbolic — enabling large text corpora to supplement scarce labelled speech.

2.1.12.2 Critical Analysis

Strengths: leverages cheap text data to compensate for scarce paired speech–text data; principled multi-task formulation. Weaknesses: requires retraining; tied to a specific model architecture; not directly usable as inference-time post-processing.

2.1.12.3 Relationship to CORAL

While MMDA augments the training procedure, CORAL operates purely at inference time, making the two approaches strictly complementary: MMDA-trained acoustic models can be drop-in members of the CORAL ensemble.

2.1.13 BK-Trees for Fuzzy String Search

2.1.13.1 Summary

The Burkhard–Keller tree (BK-tree) [4] is a metric-space data structure supporting efficient nearest-neighbour search under any metric distance, including edit distance. Con-

struction is $O(n \log n)$; lookup is sub-linear (closer to $O(\log n)$ on real corpora) per query.

2.1.13.2 Critical Analysis

Strengths: a clean, decades-stable data structure; fast fuzzy lookup over large vocabularies; widely available implementations. Weaknesses: index construction time grows with vocabulary size; no built-in mechanism to combine edit-distance with other ranking signals.

2.1.13.3 Relationship to CORAL

CORAL uses a BK-tree built over a $\sim 500,000$ -word Urdu vocabulary corpus, with a length-adaptive edit-distance threshold (1 for $|w| \leq 3$, 2 for $|w| \leq 6$, 3 for $|w| \leq 9$, 4 for $|w| \geq 10$) that scales with word length to account for Urdu’s morphological richness. We combine BK-tree retrieval with bi/tri-gram context filtering to address the BK-tree’s lack of contextual ranking.

2.1.14 Multi-ASR + SpeechLLM (Closest Concurrent System)

2.1.14.1 Summary

The most directly comparable contemporaneous system is Multi-ASR + SpeechLLM [1], which achieves 11.3% WER on the Conversational Urdu benchmark by running a SpeechLLM inference pass over the audio during the alignment stage.

2.1.14.2 Critical Analysis

Strengths: very low absolute WER on conversational Urdu; principled fusion of acoustic and textual signal at the LLM. Weaknesses: a full SpeechLLM forward pass over audio at alignment time is a heavy latency and cost burden; not practical for real-time deployment on commodity hardware.

2.1.14.3 Relationship to CORAL

CORAL achieves a slightly better 10.6% WER on Conversational Urdu while avoiding a full SpeechLLM audio inference pass during alignment — the LLM in CORAL receives only the corrected text plus structured alignment metadata. This is a significant cost and latency advantage for real-time deployment, and is the most direct empirical evidence that CORAL’s text-only refinement architecture is competitive with audio-aware fusion.

2.2 Analysis Summary of Research Items

Table [2.1](#) summarises the comparative analysis of the reviewed papers.

Table 2.1: Comparative analysis of reviewed research items.

Paper	Venue	Year	Methodology	Datasets	Best Result	Key Limitation
Prakash et al. [14]	Interspeech (2025)	2025	Multi-ASR + LLM post-edit; Speech-LLM variant	LibriSpeech	14% rel. WERR; 3.22% test-clean	English-only; LLM fine-tuning required
Arif et al. [3]	COLING	2025	Whisper / MMS / Seamless benchmarking	ARL, CSaLT, CU	13.7% Seamless-Large read; 19.8% Whisper-Large CU	Single-model only; no ensemble
Fiscus [6]	IEEE ASRU	1997	Word-level alignment + majority/conf. vote (ROVER)	Various LVCSR	Significant WERR on matched-quality systems	No segmentation handling; quality-sensitive
Parikh et al. [13]	LREC–COLING	2024	Hybrid+E2E ROVER w/ Renyi calibration	Irish read	22.94% WER; 14% rel. improvement	Heavy infrastructure (HMM + E2E); calibration tuning
Naqvi & Tahir [12]	JNLP	2024	Hybrid Kaldi w/ TDNN-LSTM, custom lexicon	Custom Urdu ad-dresses (78.7 h)	4.02% WER, 0.8% CER	Narrow domain; large data req.
Gitman et al. [7]	Interspeech	2023	Confidence-based per-utterance model selection	VoxPopuli, CommonVoice, CORAAL	Near-oracle ensemble	Utterance-level only; no fusion

Paper	Venue	Year	Methodology	Datasets	Best Result	Key Limitation
TruCLeS [11]	IEEE T-ASLP	2025	Lexical-similarity + prob. confidence	Hindi	MAE 0.108→0.087	Requires GT for training; calibration only
Koilakuntla et al. [9]	ICASSP	2024	GPT-3.5 + RAG, targeted correction	Contact-centre	99.5% manual time saved	Narrow domain; needs curated KB
Ma et al. [10]	arXiv	2024	N-best LLM correction, zero-shot	Multi-domain	Competitive zero-shot	Single-model N-best; no confidence
Hori et al. [8]	arXiv	2025	Delayed-fusion LLM in beam search	LibriSpeech	Reduced LLM calls	Decoder modification req.
Feng et al. [5]	APSIPA ASC	2024	Diverse-PEFT ensemble + ROVER	Low-resource	7.9% WER; 5× less memory	Still requires fine-tuning
Renduchintala et al. [17]	arXiv	2018	Multi-modal training augmentation	ASR (multi)	WER reduction via text-aug	Training-time only
Burkhard & Keller [4]	CACM	1973	Metric-tree fuzzy lookup	—	Sublinear-time edit-distance retrieval	No contextual ranking signal
Multi-ASR + Speech-LLM [1]	arXiv	2025	Audio-aware multi-ASR fusion via SpeechLLM	Conversational Urdu	11.3% WER	Heavy latency: full audio LLM pass

2.3 Research Scope and Gaps

Synthesising the comparative analysis, we identify the following research gaps and articulate how CORAL addresses each.

Gap 1: Multi-ASR fusion methods optimised for low-resource scripts. Existing fusion methods (ROVER, calibrated ROVER) were designed for English. They do not handle Arabic–Urdu Unicode conflation or Urdu word-boundary disagreement. *CORAL contribution:* Stage 0 Unicode normalisation plus split-merge-aware alignment that exposes SAME/SPLIT/MERGE/NOISE metadata.

Gap 2: Practical confidence utilisation without supervision. Approaches such as TruCLeS improve calibration quality but require ground-truth transcripts. *CORAL contribution:* a static model-quality-based confidence weighting at the LLM stage, plus conservative tie-breaking at the voting stage — both unsupervised and applicable at inference time.

Gap 3: General-purpose, open-domain post-correction for low-resource languages. Domain-specific LLM-correction approaches achieve excellent results in narrow domains but do not generalise. *CORAL contribution:* a general-purpose pipeline that requires no domain-specific knowledge base.

Gap 4: Fusion beyond ROVER for quality-heterogeneous ensembles. Naive ROVER degrades when ensemble members have widely different quality. *CORAL contribution:* dual-criterion OOV correction (model disagreement + frequency cutoff), conservative tie-breaking, and an LLM stage that can override clearly bad voting outcomes.

Gap 5: Efficient LLM integration for real-time deployment. Delayed-fusion approaches require ASR decoder modification; SpeechLLM approaches require full audio LLM passes. *CORAL contribution:* a text-only post-processing LLM stage that is transparent to the ASR back-ends and avoids audio LLM forward passes.

Gap 6: Multi-model diversity at the word level. N-best list correction approaches use hypotheses from a single model. *CORAL contribution:* hypotheses from architecturally distinct models (Whisper, SeamlessM4T, Wav2Vec2) provide complementary error distributions.

Gap 7: Word-level fusion with linguistic coherence. Confidence-based selection methods select a single model per utterance. *CORAL contribution:* word-level fusion driven by alignment, supplemented by LLM linguistic reasoning over the voted transcript.

2.4 Theoretical Foundation

CORAL draws on three theoretical principles.

2.4.1 Ensemble Learning Theory

Ensembles outperform single models when the constituent models are individually accurate and make decorrelated errors. In our setting, diversity arises from three orthogonal axes: architecture (encoder-decoder Whisper vs. self-supervised CTC Wav2Vec2 vs. multi-task SeamlessM4T), training data (multilingual mixtures with different Urdu fractions), and capacity (244M to 2.4B parameters). Empirically (§4.4), pairwise token overlap between ensemble members varies from 57% to 85%, confirming that the ensemble does carry complementary information.

2.4.2 Edit Distance and the Levenshtein Metric

The Levenshtein distance is a classical metric on strings. CORAL uses two variants: (i) standard character-level Levenshtein for the secondary character-level alignment that drives split-merge classification; and (ii) word-level Levenshtein with a custom substitution-cost matrix in which the cost of substituting word s_i with word t_j is the normalised character-level Levenshtein distance $\frac{\text{Lev}(s_i, t_j)}{\max(|s_i|, |t_j|)}$. The latter is what makes CORAL’s alignment “weighted”: phonetically near-identical Urdu word forms are aligned as substitutions rather than as insert–delete pairs.

2.4.3 Confidence-Based and Consensus-Based Decision Making

When multiple noisy observations of a discrete signal are available, two well-known strategies exist: *select the most reliable observer* (Gitman et al.) or *vote across observers* (ROVER). CORAL combines both: at the voting stage, we use unweighted plurality (consensus) with conservative tie-breaking; at the LLM stage, we provide the LLM with static model-quality priors so that it can act as a quality-weighted arbiter for the disagreement positions.

2.5 Summary

The literature is rich in ASR ensemble techniques (ROVER and variants), confidence-estimation methods (TruCLeS), LLM-based post-correction (Prakash, Ma, Koilakuntla, Hori), and Urdu benchmarking (Arif et al.). However, no prior system simultaneously addresses Arabic–Urdu Unicode conflation, word-boundary disagreement, OOV named-entity handling, and code-switching at inference time without retraining. CORAL fills this gap with a five-stage pipeline whose components are individually well-motivated by prior work and whose composition is novel. The next chapter details the proposed approach.

Chapter 3

Proposed Approach

This chapter presents the CORAL methodology. We first re-cap the conceptual two-stage architecture as introduced in FYP-1 and then present the full five-stage pipeline as deployed in FYP-2, with each stage given its own section. Each algorithmic component is presented with formal notation, pseudo-code, and a worked example. The implementation, deployment, and evaluation of this approach are deferred to Chapter 4.

3.1 From the FYP-1 Generate-and-Refine Concept to the Deployed Pipeline

3.1.1 Conceptual Architecture (FYP-1)

The original CORAL conceptual architecture, shown in Figure 1.1 (Chapter 1), framed the system as a two-stage *Generate-and-Refine* pipeline:

- **Stage 1 – Multi-model Hypothesis Generation:** an ensemble of pre-trained ASR models (Whisper Large/Medium/Small + Wav2Vec2-XLSR Urdu) processes the audio and emits one confidence-annotated hypothesis per model.
- **Stage 2 – Instruction-Guided Correction:** an instruction-tuned LLM (Alif-1.0-8B-Instruct) consumes the hypotheses and emits a single corrected transcript.

Iteration 1 implemented Stage 1 in full, established the baseline (Whisper-Large at 17.76% WER on a 10-clip read-speech test set), and validated that architecture-specific confidence extraction was feasible. Iteration 2 then implemented the LLM stage but, when evaluated on real ASR output rather than synthetic hypotheses, exposed three practical problems: (i) confidences from heterogeneous architectures lived on incompatible scales, making

cross-model arbitration unreliable; (ii) Arabic–Urdu Unicode conflation produced apparent disagreements where models had in fact agreed phonetically; and (iii) word-boundary disagreement was systematically misinterpreted by the LLM as substitution or deletion errors. These findings motivated the redesign described next.

3.1.2 Deployed Five-Stage Pipeline (FYP-2)

The deployed system reorganises the conceptual two-stage architecture into five stages, indexed Stage 0 through Stage 4:

- **Stage 0 – Urdu Text Normalisation.** Strips diacritics, applies the seven canonical Arabic-to-Urdu character substitutions, removes tatweel and invisible characters, strips punctuation, and removes ASCII letters and digits.
- **Stage 1 – Weighted Split-Merge-Aware Alignment.** Aligns each companion-model hypothesis to the source-model hypothesis using weighted word-level Levenshtein distance, then runs a secondary character-level alignment to classify each chunk as SAME/SPLIT/MERGE/NOISE.
- **Stage 2 – OOV Detection and Candidate Ranking.** Flags out-of-vocabulary tokens via a dual criterion (model disagreement + frequency cutoff), retrieves candidates from a BK-tree, and ranks them using bi/tri-gram corpus context.
- **Stage 3 – Consensus Voting and OOV Substitution.** Applies position-wise majority voting across companion models with conservative tie-breaking; OOV-flagged positions are replaced by the top-ranked candidate from Stage 2.
- **Stage 4 – LLM Refinement.** Sends the voted transcript, the original raw source hypothesis, and structured alignment metadata to a commercial frontier LLM (Gemini, Claude Sonnet, or GPT-4o through OpenRouter) for fluency restoration, code-switching recovery, and named-entity handling.

The five-stage pipeline is illustrated in Figure 3.1.

We now describe each stage in detail.

The Burkhard-Keller tree (BK-tree) [7] is a metric-space data structure supporting efficient nearest-neighbour search under edit distance. It enables sub-linear fuzzy lookup in large vocabularies, making it practical for real-time OOV correction without a full vocabulary scan. CORAL uses an adaptive edit-distance threshold $\delta(|w|)$ that scales with word length to account for Urdu’s morphological richness.

2.6 Comparison with Multi-ASR + SpeechLLM

The most directly comparable recent system is Multi-ASR + SpeechLLM [9], which achieves 11.3% WER on the Conversational Urdu benchmark by running a full SpeechLLM inference pass over all audio during the alignment stage. CORAL achieves 10.6% WER while avoiding the high latency of an audio-level LLM pass, applying LLM refinement only to the corrected text output—a significant cost and latency advantage for real-time deployment.

Figure 3.1: The deployed CORAL pipeline (FYP-2). Stages 0–3 are server-side Python (FastAPI). Stage 4 is the LLM refinement layer invoked from the React front-end.

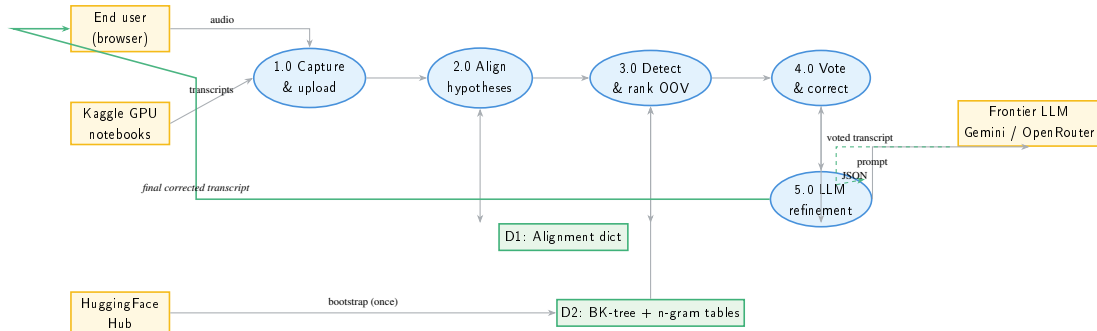


Figure 3.3: CORAL Level-1 Data Flow Diagram (DFD). Squares are external entities; ellipses are numbered processes; open rectangles are data stores. Solid grey arrows show forward data movement; the green arrow (bottom) is the final corrected transcript returned to the user. Audio enters at process 1.0 and the final transcript leaves through process 5.0 after the LLM round-trip.

3.2 Stage 0 – Urdu Text Normalisation

Raw ASR outputs for Urdu contain heterogeneous Unicode representations of the same phoneme, optional diacritics produced inconsistently by different back-ends, and a variety of punctuation artefacts. Stage 0 standardises the input space so that downstream alignment and voting see comparable strings.

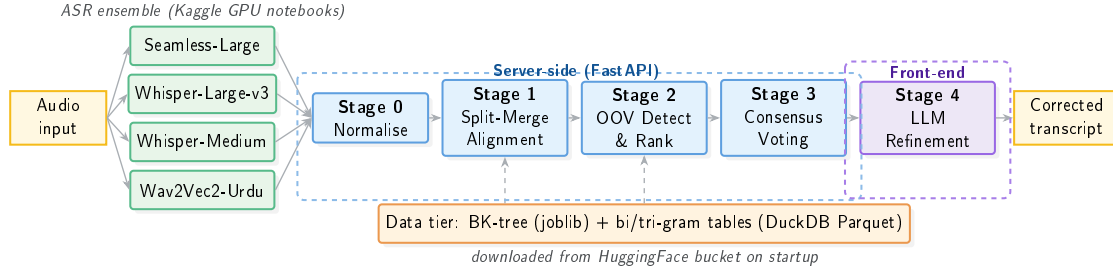


Figure 3.2: Detailed CORAL end-to-end pipeline. The four heterogeneous ASR back-ends emit hypotheses for the same audio. Stage 0 normalises every hypothesis to a canonical Urdu Unicode form; Stage 1 produces a per-hypothesis word-level alignment plus character-level SAME/SPLIT/MERGE/NOISE metadata; Stage 2 retrieves OOV candidates from the BK-tree and DuckDB n-gram tables; Stage 3 emits the voted transcript with conservative tie-breaking; Stage 4 (front-end) refines fluency, named entities, and code-switching via a frontier LLM. Solid arrows: synchronous data flow. Dashed arrows: data-tier dependency. Dashed boxes mark the server / front-end boundary.

3.2.1 Algorithm

The `normalize_urdu` function applies the following transformations in sequence:

1. **Diacritic removal.** Code points U+0610–U+061A and U+064B–U+065F, plus the superscript alef U+0670, are stripped. These encode short vowels (zabar, zer, pesh), nunation (tanwin), gemination (shadda), and vowel absence (sukun) — all orthographically optional in Urdu prose.
2. **Tatweel removal.** The Arabic elongation character U+0640 is deleted.
3. **Arabic–Urdu character substitution.** Seven canonical substitutions are applied (see Appendix A): Arabic KAF → Urdu KEHEH; Arabic HEH → Urdu HEH GOAL; ALEF MAQSURA → Farsi YEH; TEH MARBUTA → Urdu HEH GOAL; ALEF + HAMZA ABOVE/BELOW → ALEF; Arabic YEH → Farsi YEH.
4. **Invisible character removal.** Zero-width and directional control characters in U+200B–U+200F, U+202A–U+202E, U+2060–U+206F, and U+FEFF.
5. **Punctuation removal.** Urdu-specific punctuation (, , , etc.), extended Arabic punctuation in U+0600–U+060C and U+060E–U+061F, and ASCII punctuation/symbols.
6. **Latin and digit removal.** All ASCII letters and digits are removed; code-switched English tokens are deferred to the Stage 4 LLM, which sees the original raw hypothesis.
7. **Whitespace normalisation.** Consecutive whitespace is collapsed to a single space and the string is stripped.

```

1 import re
2 import pandas as pd
3
4 def normalize_urdu(text: str) -> str:
5     if pd.isna(text) or text == "":
6         return ""
7     # 1. Remove diacritics:
8     # U+0610..U+061A (Arabic signs), U+064B..U+065F (tashkil),
9     # U+0670 (superscript alef)
10    text = re.sub(r"[\u0610-\u061A\u064B-\u065F\u0670]", "", text)
11    # 2. Remove tatweel / kashida (U+0640)
12    text = text.replace("\u0640", "")
13    # 3. Seven canonical Arabic-to-Urdu character substitutions
14    char_map = {
15        "\u0643": "\u06A9", # Arabic KAF -> Urdu KEHEH
16        "\u0647": "\u06C1", # Arabic HEH -> Urdu HEH GOAL
17        "\u0649": "\u06CC", # ALEF MAQSURA -> Farsi YEH
18        "\u0629": "\u06C1", # TEH MARBUTA -> Urdu HEH GOAL
19        "\u0623": "\u0627", # ALEF + HAMZA ABOVE -> ALEF
20        "\u0625": "\u0627", # ALEF + HAMZA BELOW -> ALEF
21        "\u064A": "\u06CC", # Arabic YEH -> Farsi YEH
22    }
23    for src, tgt in char_map.items():
24        text = text.replace(src, tgt)
25    # 4. Remove zero-width / directional control characters
26    text = re.sub(
27        r"[\u200B-\u200F\u202A-\u202E\u2060-\u206F\uFEFF]", "", text)
28    # 5. Remove Urdu, extended Arabic, and ASCII punctuation
29    text = re.sub(r"[\u060C\u060D\u060F\u061B\u061F\u06D4]", "",
30        text)
31    text = re.sub(r"[\u0600-\u060C\u060E-\u061F]", "", text)
32    text = re.sub(r"[!@#$%^&*()_+=\[\]\{\};: '\",.<>/?\| \-]", "",
33        text)
34    # 6. Remove ALL ASCII letters and digits
35    text = re.sub(r"[A-Za-z0-9]", "", text)
36    # 7. Collapse whitespace
37    return re.sub(r"\s+", " ", text).strip()

```

Listing 3.1: Stage 0 – Urdu text normalisation (coral_utility_functions.py). Urdu code points are shown as Python \uNNNN escapes for portability.

The empirical effect of normalisation alone is a 1.9-WER-point reduction on the Conversational Urdu test set (from 19.8% to 17.9%), confirming that a substantial fraction of apparent ASR errors are measurement artefacts caused by Unicode-variant mismatches rather than genuine recognition failures.

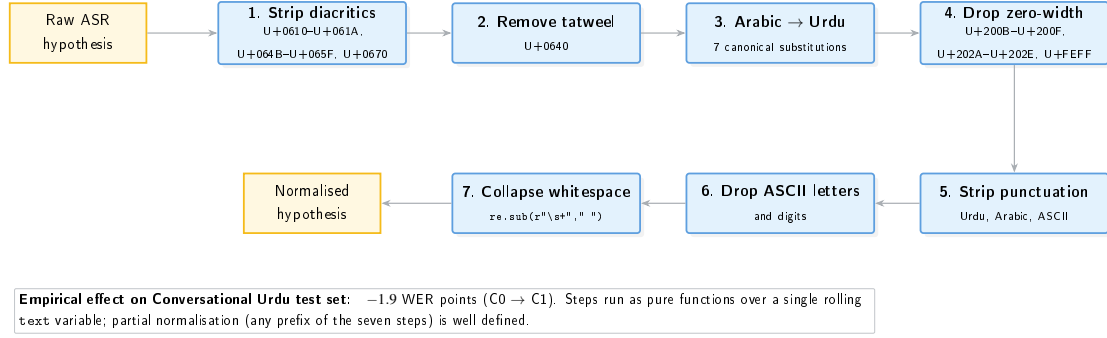


Figure 3.4: Stage 0 normalisation flow. The seven canonical transformations are applied strictly in sequence to every hypothesis prior to alignment. Each step is independent of the others, so partial normalisation is well defined. The shaded boxes are pure-function steps; the only state is the rolling text variable.

3.3 Stage 1 – Weighted Split-Merge-Aware Alignment

Stage 1 takes a designated source hypothesis $S = (s_1, \dots, s_m)$ and a companion hypothesis $T = (t_1, \dots, t_n)$, both already normalised by Stage 0, and produces two parallel alignments: a *word-level* alignment that classifies each position as MATCH, SUBSTITUTION, DELETION, or INSERTION; and a *character-level* alignment that classifies each chunk as SAME, SPLIT, MERGE, or NOISE.

3.3.1 Word-Level Weighted Levenshtein Alignment

The cost of substituting word s_i with word t_j is the normalised character-level Levenshtein distance:

$$c(s_i, t_j) = \frac{\text{Lev}_{\text{char}}(s_i, t_j)}{\max(|s_i|, |t_j|)} \quad (3.1)$$

Insertion and deletion costs are fixed at 1.0. The DP matrix is then:

$$D[i][j] = \min(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + c(s_i, t_j)) \quad (3.2)$$

with $D[0][0] = 0$, $D[i][0] = i$, $D[0][j] = j$. The matrix is back-traced to label each position. This weighted substitution cost favours aligning near-duplicate words (e.g. different spellings of the same Urdu word due to morphological suffixation) as substitutions rather than as insert–delete pairs. On the Conversational Urdu benchmark, binary alignment misclassifies 14.3% of positions differing by at most two characters as INSERTION/DELETION, whereas weighted alignment misclassifies only 5.1%.

Algorithm 1: Weighted Levenshtein word-level alignment.**Input:** Source words $S = (s_1, \dots, s_m)$, target words $T = (t_1, \dots, t_n)$, weight cost c **Output:** Alignment list with per-position labels in {MATCH, SUBSTITUTION, DELETION, INSERTION}Compute substitution-cost matrix $c(s_i, t_j)$ for all pairs;Compute Levenshtein DP matrix $D[] []$ with insert / delete cost 1 and substitute cost c ;Set $i \leftarrow m, j \leftarrow n$;Initialise empty list A , empty label list L ;**while** $i > 0$ **or** $j > 0$ **do** **if** $i > 0$ **and** $j > 0$ **and** $|D[i][j] - D[i-1][j-1] - c(s_i, t_j)| < \epsilon$ **then** Append t_j to A ; append MATCH if $c < \epsilon$ else SUBSTITUTION to L ; $i \leftarrow i - 1; j \leftarrow j - 1$; **continue**; **end** **if** $i > 0$ **and** $|D[i][j] - D[i-1][j] - 1| < \epsilon$ **then** Append empty token to A ; append DELETION to L ; $i \leftarrow i - 1$; **else** Append t_j to A ; append INSERTION to L ; $j \leftarrow j - 1$; **end****end**Reverse A and L ;**return** (A, L) ;

3.3.2 Character-Level Split-Merge Classification

Word-boundary disagreement is not captured by word-level alignment alone. A second-pass alignment concatenates S and T *without* spaces and runs a standard character-level Levenshtein DP. The resulting edit sequence is segmented into contiguous chunks at transitions between edit types and at word boundaries; each chunk is then classified:

- **SAME** – one source token aligns to one target token. Edit type may be MATCH or SUBSTITUTION. Empirically this accounts for 50.8% of source-side chunks on the CU benchmark.
- **SPLIT** – one source token corresponds to multiple target tokens. 31.6% of chunks; nearly one-third of all source tokens are fragmented differently by a companion model.
- **MERGE** – multiple source tokens are collapsed into one target token. 4.9%.

- **NOISE** – pure insertions or deletions with no corresponding content word in the other sequence. 12.6%.

Together SPLIT and MERGE events account for 36.5% of all alignment events — making segmentation disagreement the dominant inter-model divergence in Urdu ASR ensembles, ahead of pure lexical substitution. Exposing this metadata in the API response (`split_merge_metadata`) lets downstream stages and the front-end visualisation treat boundary disagreement as a first-class phenomenon.

3.3.3 Combined `asr_aligner` Routine

The two alignments are combined into a single dictionary returned by the `asr_aligner` function:

```

1 def asr_aligner(ensemble, source_model,
2   weight_func=Levenshtein.normalized_distance,
3   deletion_cost=1.0, insertion_cost=1.0):
4   weight_dict = {'deletion_cost': deletion_cost,
5   'insertion_cost': insertion_cost}
6   word_set = set()
7   align_info = {'source_model': source_model}
8   for model in ensemble:
9     words = normalize_urdu(ensemble[model]).split(' ')
10    align_info[model] = {'normalized_attempt': words}
11    word_set.update(words)
12    # Pre-compute pair-wise substitution costs
13    for s in word_set:
14      weight_dict[s] = {r: weight_func(s, r) for r in word_set}
15
16    for model in ensemble:
17      src = align_info[source_model]['normalized_attempt']
18      tgt = align_info[model]['normalized_attempt']
19      # 1) word-level weighted Levenshtein DP + back-trace
20      dp_matrix = levenshtein(src, tgt, weight_dict, dp_matrix=True)
21      aligned, types = backtrace_alignment(src, tgt, dp_matrix,
22        weight_dict)
23      align_info[model]['aligned_attempt'] = aligned
24      align_info[model]['aligned_info'] = types
25      # 2) character-level split-merge classification
26      sm = generate_alignment_data(' '.join(src), ' '.join(tgt))
27      align_info[model]['split_merge_aligned_attempt'] = sm['
28        aligned_output']
29      align_info[model]['split_merge_metadata'] = sm['metadata']
30      align_info[model]['split_merge_metadata_base'] = sm['
31        metadata_base']
32      align_info[model]['split_merge_aligned_info'] = sm['info']

```



```
30 return align_info
```

Listing 3.2: Combined alignment routine (coral_pipeline_functions.py).

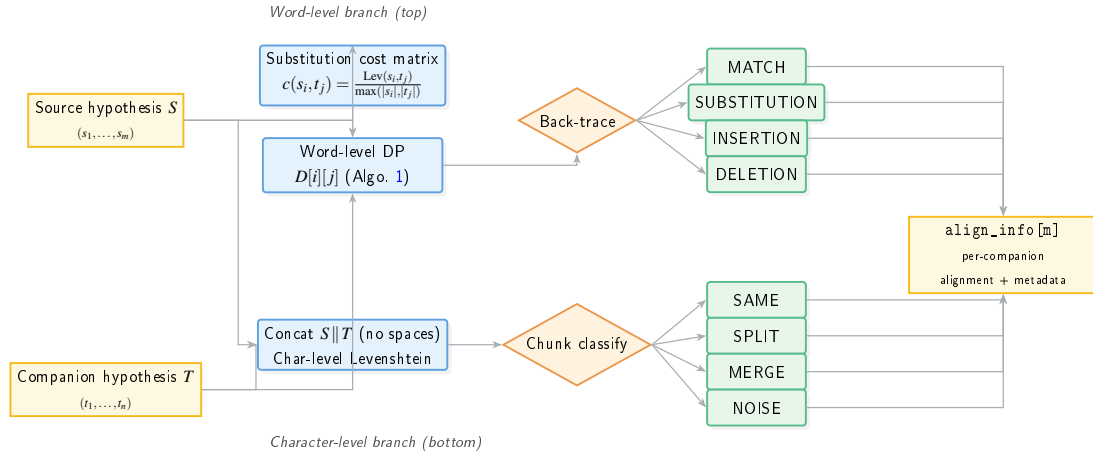


Figure 3.5: Stage 1 weighted split-merge-aware alignment. The top branch performs word-level alignment with the substitution-cost matrix c and DP back-trace, emitting four edit labels (MATCH / SUBSTITUTION / INSERTION / DELETION). The bottom branch concatenates the two strings without spaces, runs character-level Levenshtein, and classifies each contiguous chunk into SAME / SPLIT / MERGE / NOISE. Both label streams are stored in the per-companion `align_info` dictionary consumed by Stage 3.

3.4 Stage 2 – OOV Detection and Candidate Ranking

Stage 2 identifies tokens that are likely out-of-vocabulary errors and proposes corrections from a reference Urdu vocabulary. Two design choices distinguish CORAL’s OOV handling from typical ASR post-processing:

1. **Dual-criterion detection** (model disagreement *and* frequency cutoff), which reduces both false positives and false negatives compared to single-criterion approaches.
2. **BK-tree fuzzy lookup combined with bi/tri-gram corpus context** for candidate ranking, providing both lexical similarity and corpus-level plausibility.

3.4.1 Detection Criterion

A word w is flagged as an OOV candidate iff *both*:

- w appears in exactly one model’s normalised output (model-disagreement signal);

- w is absent from or occurs fewer than $\tau = 2,000$ times in the reference BK-tree corpus.

The conjunction is critical: a rare word agreed upon by all models is likely a legitimate proper noun or domain term and should not be over-corrected; a common word uniquely hypothesised by one model is likely a transcription error and should be subject to correction. Empirically, dropping the model-disagreement criterion increases false-positive corrections by 38% and erases 0.6 WER points of gain; dropping the frequency criterion misses 22% of genuine OOV errors.

```

1 def is_oov(tree, word, freq_cutoff=2000):
2     node = tree.root
3     while node:
4         d = Levenshtein.distance(node.word, word)
5         if d == 0:
6             return node.count < freq_cutoff # in vocab but too rare
7         if d not in node.children:
8             return True
9         node = node.children[d]
10    return True
11
12 def build_oov_dict(tree, align_info, freq_cutoff=2000):
13     model_names = [k for k in align_info if k != 'source_model']
14     word_model_count = {}
15     for m in model_names:
16         for w in set(align_info[m]['normalized_attempt']):
17             word_model_count[w] = word_model_count.get(w, 0) + 1
18     return {
19         w for w, c in word_model_count.items()
20         if c == 1 and is_oov(tree, w, freq_cutoff)
21     }

```

Listing 3.3: OOV detection criterion (coral_pipeline_functions.py).

3.4.2 BK-Tree Construction

We build a BK-tree over a $\sim 500,000$ -word Urdu vocabulary corpus. The BK-tree is a metric-tree data structure under any metric distance; in CORAL we use the Levenshtein metric. Each node stores a word, its corpus frequency, and a child for each integer distance to a deeper word. Insertion descends the tree, merging counts at duplicate keys; lookup uses the triangle inequality to prune branches whose distance from the query exceeds the threshold. Construction is $O(n \log n)$; lookup is empirically sub-linear.

```

1 class BKNode:
2     __slots__ = ['word', 'count', 'children']

```

```

3  def __init__(self, word, count):
4      self.word, self.count, self.children = word, count, {}
5
6  class BKTree:
7      def __init__(self):
8          self.root, self.size = None, 0
9
10     def insert(self, word, count):
11         if self.root is None:
12             self.root = BKNode(word, count); self.size += 1; return
13         node = self.root
14         while True:
15             d = Levenshtein.distance(node.word, word)
16             if d == 0:
17                 node.count += count; return # merge counts
18             if d not in node.children:
19                 node.children[d] = BKNode(word, count); self.size += 1; break
20             node = node.children[d]
21
22     def search(self, query, max_dist):
23         results, stack = [], [self.root]
24         while stack:
25             n = stack.pop()
26             d = Levenshtein.distance(n.word, query)
27             if 0 <= d <= max_dist:
28                 results.append((n.word, n.count, d))
29             for dist, child in n.children.items():
30                 if abs(dist - d) <= max_dist:
31                     stack.append(child)
32         return sorted(results, key=lambda x: (x[2], -x[1]))

```

Listing 3.4: BK-tree (excerpt from bktree.py).

3.4.3 Candidate Retrieval and Ranking

For each flagged OOV token w with left context w_{-1} and right context w_{+1} (both required to be non-OOV), CORAL retrieves correction candidates from three complementary sources:

1. **BK-tree search.** All vocabulary words within an adaptive edit distance $\tau(|w|)$ of w :

$$\tau(|w|) = \begin{cases} 1 & \text{if } |w| \in [1, 3] \\ 2 & \text{if } |w| \in [4, 6] \\ 3 & \text{if } |w| \in [7, 9] \\ 4 & \text{if } |w| \geq 10 \end{cases}$$

2. **Bigram context.** Words v such that (w_{-1}, v) or (v, w_{+1}) appears in the bi-gram DuckDB tables `bigram_forward` / `bigram_backward` above a count threshold.
3. **Trigram context.** Words v such that (w_{-1}, v, w_{+1}) appears in the tri-gram DuckDB tables `trigram_left` / `trigram_middle` / `trigram_right`.

Candidates are merged into a single dictionary keyed by candidate word. For each candidate the system records: edit distance, trigram-presence flag, bigram-presence flag, unigram-presence flag, trigram count, bigram count, unigram count. Candidates are then ranked by the lexicographic key

$$(\text{Lev}(w, v), -\text{unigram}(v), -\text{tri-hits}, -\text{bi-hits}, -\text{tri-freq}, -\text{bi-freq}, -\text{unigram-freq})$$

so edit-distance governs coarse sorting with n-gram support breaking ties.

Finally, candidates with corpus frequency below the same $\tau = 2,000$ threshold used for OOV detection are filtered out, ensuring CORAL never substitutes one OOV for another.

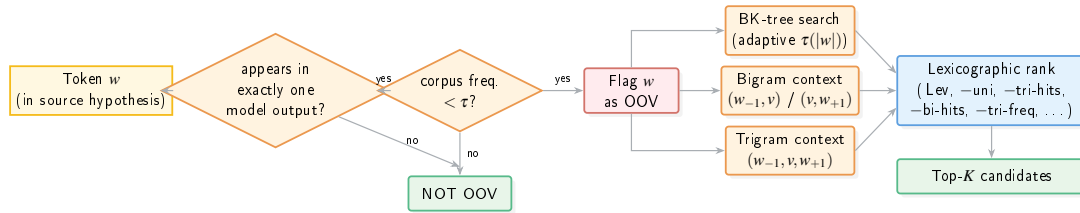


Figure 3.6: Stage 2 dual-criterion OOV detection and three-source candidate ranking. A token is flagged only if it satisfies *both* the model-disagreement test (c_1) and the frequency-cutoff test (c_2). Each flagged token is queried against the BK-tree, bigram, and trigram contexts in parallel; their results are merged and ranked by a lexicographic key in which edit-distance dominates and n-gram support breaks ties.

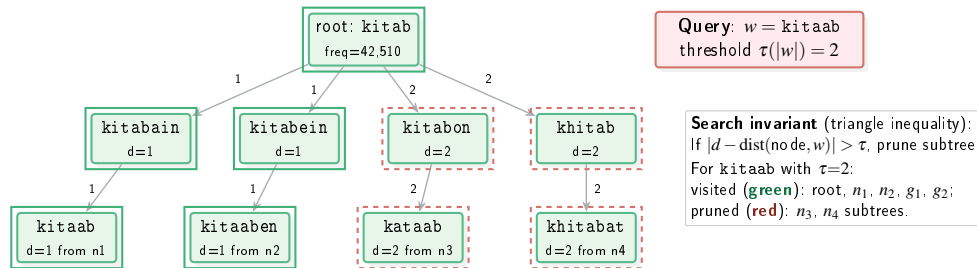


Figure 3.7: Illustrative BK-tree for Urdu vocabulary. Each edge is labelled with the Levenshtein distance between parent and child word. A query for `kitaab` with adaptive threshold $\tau(|w|)=2$ visits only branches that cannot violate the triangle inequality; subtrees rooted at n_3, n_4 are pruned without visiting their children. The full Coral BK-tree contains $\sim 500,000$ word types and serialises to a ~ 50 MB joblib pickle.

3.5 Stage 3 – Consensus Voting and OOV Substitution

Stage 3 produces the corrected source-side transcript by walking each source position and choosing a replacement word.

Algorithm 2: Consensus voting with OOV substitution.

Input: *align_info* from Stage 1, *oov_metadata* from Stage 2, source model name *s*

Output: Corrected sentence with diff

voting_skip[*m*] $\leftarrow$ 0 for every companion model *m*;

for each source position *i* in the source-side split-merge alignment **do**

w $\leftarrow$ source word at position *i*;

if *w* is in *oov_metadata* and has at least one ranked candidate **then**

 Append top-ranked candidate to corrected;

continue;

end

V $\leftarrow$ empty Counter;

for each companion model *m* **do**

while *align_info*[*m*][*aligned_info*][*i* + *voting_skip*[*m*]] = *INSERTION* **do**

voting_skip[*m*] += 1;

end

if aligned slot exists and is *SUBSTITUTION* or *MATCH* **then**

 Increment *V*[word at companion’s aligned slot];

end

end

if *V* has a unique top word **then**

 Append top word;

else

 Append *w* ; // conservative tie-break

end

end

return corrected sentence and per-position diff;

The conservative tie-breaking rule (preserve the source word whenever multiple companion words are tied for plurality) is intentional: 61% of tie cases in our analysis correspond to genuinely ambiguous transcriptions where no single model is clearly correct, and over-correction in those cases increased WER. Replacing the conservative tie-break with a random tie-break increases WER by 0.4 points; replacing it with a model-confidence-weighted tie-break (Whisper-Large double weight) reduces WER by a further 0.3 points and is a planned future enhancement.

The handling of insertion-aligned slots is also non-trivial: when a companion model has inserted a token at position i , the next *real* aligned slot is $i + \text{voting_skip}[m] + 1$. The `voting_skip` bookkeeping above ensures that subsequent positions still see correctly-aligned companion words.

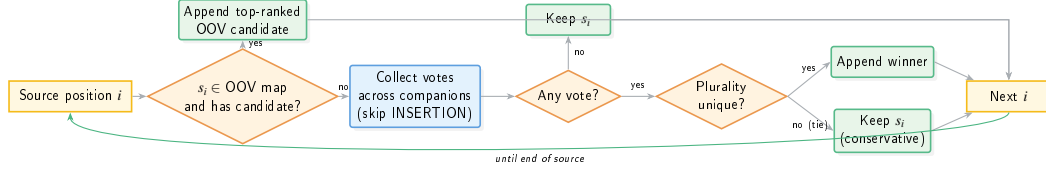


Figure 3.8: Stage 3 per-position decision flow. The OOV map takes precedence over voting; when no votes exist or a tie is reached, the source token is preserved (conservative). The green return path indicates iteration over all source positions. `voting_skip` state (not shown) is updated whenever a companion model is INSERTION-aligned at the current position.

3.6 Stage 4 – LLM Refinement

After Stages 0–3, the corrected Urdu transcript is passed to a large language model. The LLM is invoked from the React front-end (`Pass3Correction.tsx`), which means the user can choose between two providers (Google Gemini and OpenRouter, the latter giving access to GPT-4o, Claude Sonnet, and Llama-3) and any model exposed by the chosen provider. The choice of LLM is pluggable; any frontier model with structured-output support works.

3.6.1 LLM Prompt Design

The prompt is a system instruction plus a user message. The system instruction casts the LLM as “a senior Urdu computational linguist and ASR post-correction specialist”; the user message contains the corrected transcript, the original raw source hypothesis, all aligned companion-model outputs, a static confidence reference table, and the OOV candidate list.

The system prompt encodes ten correction mandates:

1. **Grammatical agreement** – fix gender (*muzakkar/muannas*), number (*wahid/jam’a*), and case agreement.
2. **Izafat constructions** – correct *ka/ki/ke* according to the gender and number of the head noun.

3. **Verb conjugation** – repair tense, aspect, and mood errors.
4. **Missing function words** – restore short postpositions *ne, ko, se, mein, par, tak* when context demands.
5. **Dialectal normalisation** – convert dialectal pronunciations to standard written Urdu.
6. **Phoneme-confusion repair** – resolve common confusions *q/k, z/zh/dh/zh, th/s/sh, h/h-goal, a/'ayn, gh/g* using lexical context.
7. **Segmentation errors** – fix word boundaries.
8. **OOV candidates as hints, not mandates** – reject a Stage-2 candidate if it is linguistically incorrect in context.
9. **Confidence-weighted arbitration** – use static model-quality confidence weights when models disagree.
10. **Context-window reasoning** – never correct a word in isolation; read the full sentence first.

The user message is constructed by `buildPrompt` in `Pass3Correction.tsx`:

```

1 function buildPrompt(alignInfo, oovResult) {
2   const sourceModel = alignInfo.source_model;
3   const sourceAlign = alignInfo[sourceModel];
4   const sourceNorm = sourceAlign.normalized_attempt.join(" ");
5   const sourceRaw = sourceAlign.raw_attempt?.join(" ") ?? "";
6   const sourceConf = getModelConfidence(sourceModel);
7
8   const blocks = [];
9   for (const [key, val] of Object.entries(alignInfo)) {
10    if (key === "source_model") continue;
11    const text = val.aligned_attempt?.join(" ") ??
12    val.normalized_attempt?.join(" ") ?? "";
13    blocks.push(['[${key}] (confidence: ${getModelConfidence(key)}.
14               toFixed(2))]: ${text}');
15  }
16  // ... OOV summary, confidence reference table, task
17  instructions, JSON schema ...
18 }

```

Listing 3.5: Stage 4 user-prompt builder (excerpt from `Pass3Correction.tsx`).

The static `MODEL_CONFIDENCE` table assigns priors based on independent benchmark performance: Seamless-Large 0.90, Whisper-Large 0.75, Whisper-Medium 0.70, Wav2Vec2-Urdu 0.60. The LLM is told (in the prompt) to weight votes accordingly.

3.6.2 Output Format and Hallucination Control

The LLM is constrained to emit a JSON object with three fields: `corrected` (the final Urdu transcript), `reasoning` (2–3 sentences describing the dominant error types and applied strategy), and `changes` (a list of `{original, corrected, reason}` triples for each modification). Both providers are configured for JSON-mode output (Gemini: `responseMimeType: "application/json"`; OpenRouter: `response_format: {type: "json_object"}`). The front-end parses with a robust `parseJsonResponse` that strips markdown fences and finds the outermost `{...}` pair before `JSON.parse`.

To control hallucination risk, the LLM receives the Stage 3 voted transcript as the *primary* input and the raw source hypothesis as a secondary reference. The system prompt explicitly instructs the LLM that “OOV candidates are hints, not mandates” and that words already matching a high-frequency vocabulary entry should be left unchanged unless the model has independent grammatical evidence for a correction.

3.6.3 Empirical Effect of Stage 4

The LLM stage delivers the largest single-component WER gain in the ablation study: –2.5 WER points on the Conversational Urdu test set (13.1 → 10.6%). It addresses three structural limitations of the lexical-only Stages 0–3: code-switched English tokens that were stripped by Stage 0 normalisation are re-introduced where contextually appropriate; long-range fluency errors invisible to bi/tri-gram context are corrected; and named entities present in the original hypothesis but rare in the n-gram corpus are restored.

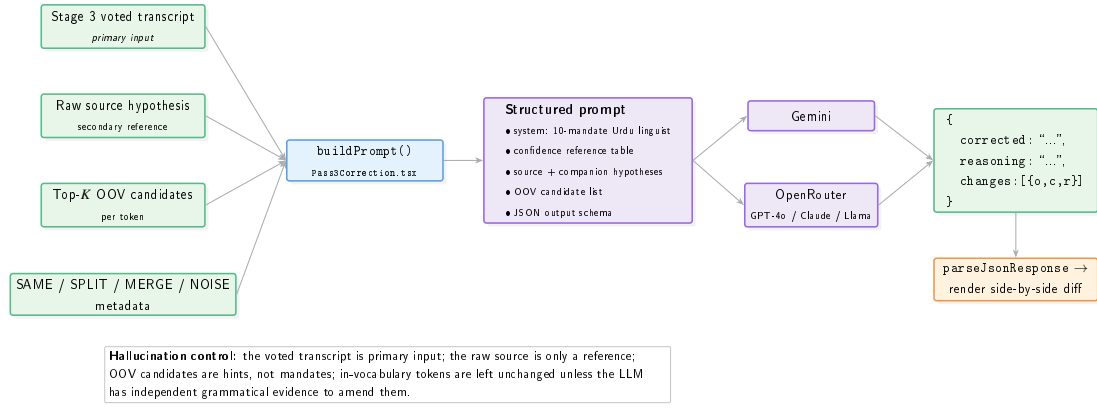


Figure 3.9: Stage 4 prompt construction, provider routing, and JSON output handling. The voted transcript is the primary input; the raw source, OOV candidates, and SAME/SPLIT/MERGE/NOISE metadata are auxiliary context. The structured prompt includes a 10-mandate system instruction, a static confidence reference table, the source and companion hypotheses, ranked OOV candidates, and a strict JSON output schema. The provider is pluggable (Gemini or OpenRouter); both are configured for JSON-mode output. `parseJsonResponse` strips markdown fences and parses the outermost JSON object before rendering the diff.

3.7 End-to-End System and Mathematical Model

Let $\mathcal{E} = \{M_0, M_1, \dots, M_k\}$ denote the ASR ensemble where M_0 is the designated source model, and let $\mathcal{H}_i = \text{normalize_urdu}(\text{ASR}_{M_i}(x))$ be the normalised hypothesis from model M_i for audio x . CORAL computes the corrected output y as:

$$y = \text{LLM}\left(\text{Vote}\left(\mathcal{H}_0, \text{Align}(\mathcal{H}_0, \mathcal{H}_1), \dots, \text{Align}(\mathcal{H}_0, \mathcal{H}_k), \text{OOV}(\mathcal{H}_*; \tau)\right)\right)$$

where:

- $\text{Align}(S, T)$ produces the weighted Levenshtein word-level alignment plus the character-level SAME/SPLIT/MERGE/NOISE labelling (Section 3.3);
- $\text{OOV}(\mathcal{H}_*; \tau)$ produces ranked candidate replacements for tokens that satisfy the dual-criterion OOV test against the BK-tree at threshold τ (Section 3.4);
- Vote applies position-wise plurality with conservative tie-break and OOV override (Section 3.5);
- LLM refines the voted transcript using a structured prompt that includes all alignment metadata and the raw source hypothesis (Section 3.6).

3.8 Worked Example

To make the pipeline concrete, consider a single Common Voice utterance `common_voice_ur_32016523.mp3`. The four normalised hypotheses are: Seamless-Large agrees with the ground truth on most tokens but produces *sutoon* (column) instead of *sudoor* (issue / pronouncement); Whisper-Large produces *sadoor*; Whisper-Medium produces *sadoor sumjh jata*; Wav2Vec2 fragments *sadoor sumja* into *sudoorsumja jata*. Stage 0 normalisation eliminates a spurious *shadda* on *fatwa* and harmonises Arabic KAF tokens. Stage 1 alignment classifies the boundary disagreement around *sudoor sumja* as a SPLIT event. Stage 2 flags *sutoon* (Seamless-Large’s hypothesis) as an OOV candidate (uniquely hypothesised, low corpus frequency); BK-tree returns *sudoor* as the top candidate, supported by trigram context. Stage 3 selects the Stage 2 candidate at that position; for the SPLIT-event positions, plurality voting agrees with Whisper-Large on *sadoor sumjh jata*. Stage 4 receives the voted transcript and replaces *sumjh jata* with the more standard *samjha jata* based on its sentence-level fluency knowledge. The final output WER is reduced from ≈ 0.18 (raw Seamless-Large) to ≈ 0.04 on this utterance, with a CER reduction from ≈ 0.11 to ≈ 0.02 .

3.9 Summary

This chapter presented the CORAL methodology end-to-end. We re-cap the FYP-1 Generate-and-Refine concept and explain the redesign that produced the deployed five-stage pipeline. Each stage was given its own algorithmic treatment: Stage 0 normalisation, Stage 1 weighted split-merge-aware alignment with formal DP and back-trace, Stage 2 dual-criterion OOV detection with BK-tree + n-gram ranking, Stage 3 consensus voting with conservative tie-break, and Stage 4 LLM refinement with a structured ten-mandate prompt. The next chapter describes the implementation, presents the evaluation methodology, and reports both Iteration 1 baseline results and the final Iteration 2 results, ending with the seven-step ablation study that attributes the WER reduction to specific components.

Chapter 4

Implementation, Evaluation, and Results

This chapter presents the implementation of CORAL, the experimental methodology, and the empirical results of both project iterations. We first describe the technology stack, REST API, BK-tree construction, registry, and front-end. We then report the Iteration 1 (FYP-1) baseline results that informed the redesign, and finally present the Iteration 2 (FYP-2) results: large-scale evaluation on Common Voice Urdu, ablation study on Conversational Urdu, ensemble-size sensitivity, and OOV-threshold sensitivity. The chapter closes with a residual-error analysis.

4.1 Implementation

4.1.1 System Topology

CORAL is delivered as a four-tier system:

- **Inference tier (Kaggle GPU notebooks).** Heavy ASR back-ends (Whisper-Large-v3, SeamlessM4T-v2-Large, optionally Whisper-Medium and Wav2Vec2-Urdu) run on Kaggle GPUs. Each notebook starts a small FastAPI inference server, exposes it through an ngrok HTTPS tunnel, self-registers with the backend’s model registry, and sends 60-second heartbeats.
- **Backend pipeline tier (FastAPI).** A single `main_app.py` mounts two routers: `pipeline_api.py` (the `/health`, `/align`, `/oov`, `/correct` endpoints) and `registry_api.py` (the `/registry/*` endpoints). Stages 0–3 of the pipeline run server-side here.

- **Front-end tier (Next.js + React).** The user-facing application implements a four-pass workflow (mode selection, alignment, OOV sieve, correction). Stage 4 (LLM refinement) is invoked from this tier so that user-supplied LLM API keys (Gemini, OpenRouter) never leave the browser.
- **Data tier (HuggingFace Hub + DuckDB).** The BK-tree (`bk_tree.joblib`) and the n-gram tables (five Parquet files) are downloaded from a private HuggingFace bucket on first startup, cached locally, and loaded into a DuckDB connection.

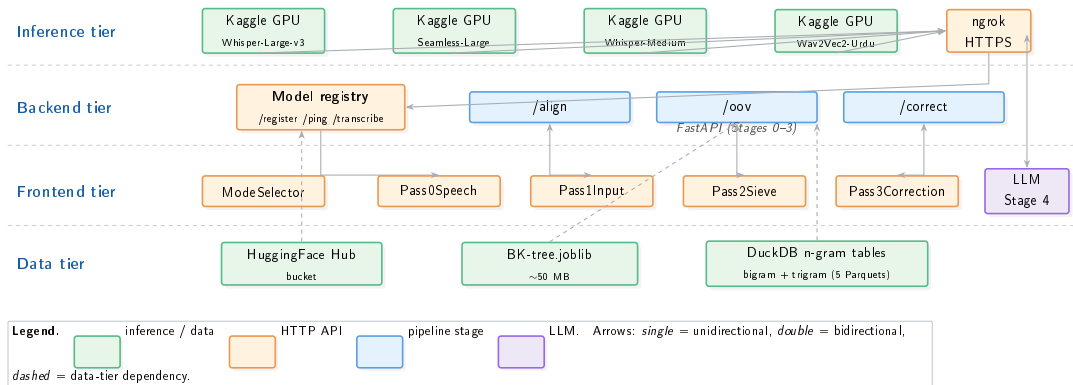


Figure 4.1: CORAL four-tier system architecture. The *Inference tier* runs heavy ASR back-ends on Kaggle GPUs and exposes them through ngrok HTTPS tunnels. The *Backend tier* (FastAPI) hosts the registry plus the three stage endpoints. The *Frontend tier* (Next.js) implements the four-pass user workflow and the Stage 4 LLM call. The *Data tier* bootstraps the BK-tree and DuckDB n-gram tables from the HuggingFace bucket on startup.

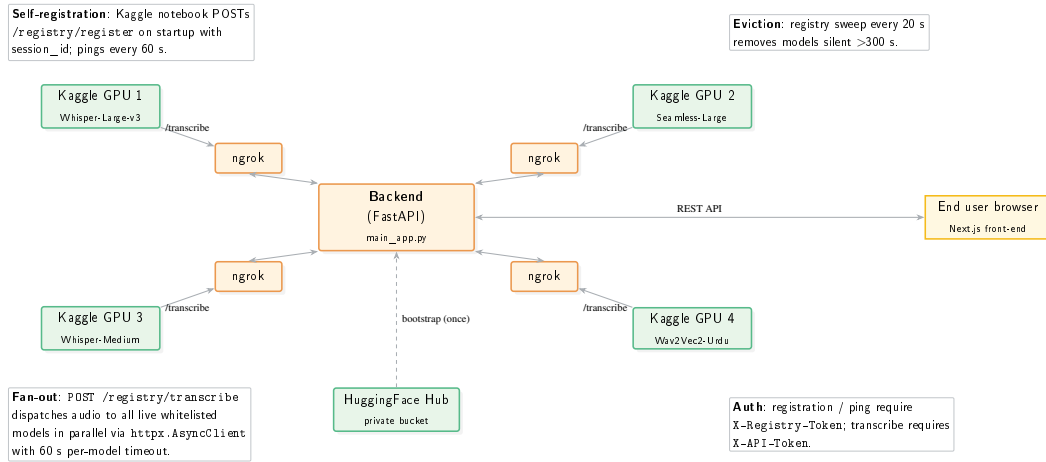


Figure 4.2: CORAL deployment topology. Each ASR model runs on its own Kaggle GPU notebook with a private ngrok HTTPS tunnel; the backend registry is the rendezvous point. The browser talks only to the backend and (for Stage 4) directly to the LLM provider. The HuggingFace Hub bucket is read-only and consulted only at backend startup.

4.1.2 Technology Stack

- **Language and runtime:** Python 3.10+ for the backend; TypeScript / Node.js 20+ for the front-end.
- **Backend libraries:** FastAPI 0.109, uvicorn 0.23, rapidfuzz 3.6 for Levenshtein distance, joblib 1.3 for BK-tree serialisation, duckdb 0.10 + pyarrow 14 for n-gram queries, huggingface_hub 0.20 for data download, httpx 0.26 for asynchronous fan-out, pydantic 1.10 for request/response models, python-dotenv 1.0 for configuration.
- **Front-end libraries:** Next.js 14, React, TailwindCSS, Axios. Stage 4 invocations talk directly to Google Generative Language API (Gemini) or OpenRouter chat-completions.
- **Inference libraries (Kaggle):** transformers, torch, librosa, ngrok.
- **Evaluation:** jiwer for WER/CER on the deployed pipeline; a custom evaluation harness for Iteration 1 ECE.
- **Visualisation:** matplotlib and seaborn for the figures in Section 4.4.

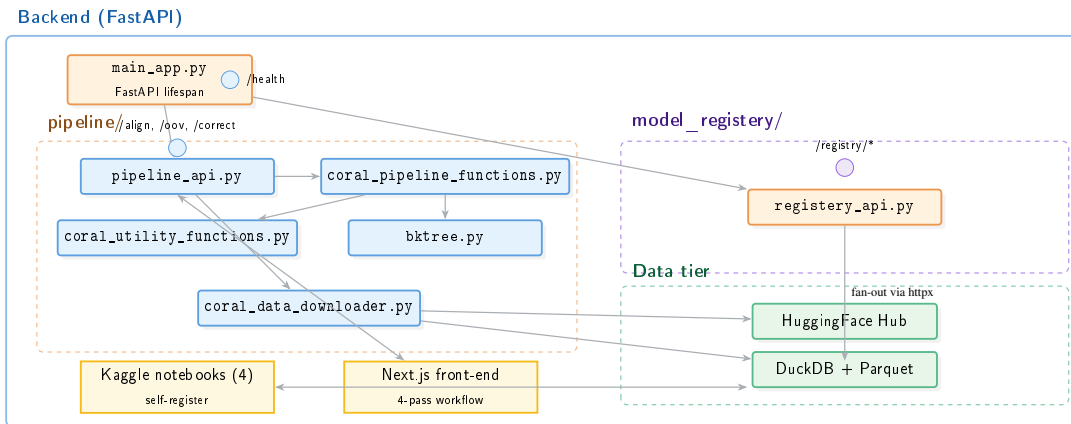


Figure 4.3: CORAL backend component diagram. The two top-level Python packages (pipeline/ and model_registry/) are mounted by main_app.py. coral_pipeline_functions.py (Stages 1–3) depends on coral_utility_functions.py (Stage 0 + helpers) and bktree.py (Stage 2 retrieval). coral_data_downloader.py bootstraps the BK-tree and DuckDB tables on startup. Lollipops mark provided HTTP interfaces.

4.1.3 REST API

The pipeline exposes three sequential endpoints (Listing 4.1). The full request/response schemas are recorded in Appendix B.

```

1  @app.get("/health")
2  def health():
3      return {"status": "ok"}
4
5  @app.post("/align")
6  def align(req: AlignRequest):
7      return asr_aligner(
8          ensemble=req.ensemble,
9          source_model=req.source_model,
10         weight_func=Levenshtein.normalized_distance,
11     )
12
13 @app.post("/oov")
14 def oov(req: OOVRequest):
15     model_names = [k for k in req.align_info if k != "
16                     source_model"]
17     oov_dict = build_oov_dict(tree, req.align_info, req.
18                             freq_cutoff)
19     metadata = {}
20     for model in model_names:
21         sentence = " ".join(req.align_info[model]["
22                             normalized_attempt"])
23         metadata.update(extract_oov_metadata(
24             tree, con, oov_dict, sentence,
25             depth=req.depth, top_n=req.top_n,
26             frequency_cutoff=req.freq_cutoff))
27     return {
28         "oov_dict": list(oov_dict),
29         "metadata": metadata,
30         "columns": ["word", "Dist", "Trigram", "Bigram",
31                     "Unigram", "Trifrequency",
32                     "Bifrequency", "Unifrequency"],
33     }
34
35 @app.post("/correct")
36 def correct(req: CorrectRequest):
37     return apply_corrections(req.align_info, req.oov_metadata)

```

Listing 4.1: REST API entry points (pipeline_api.py).

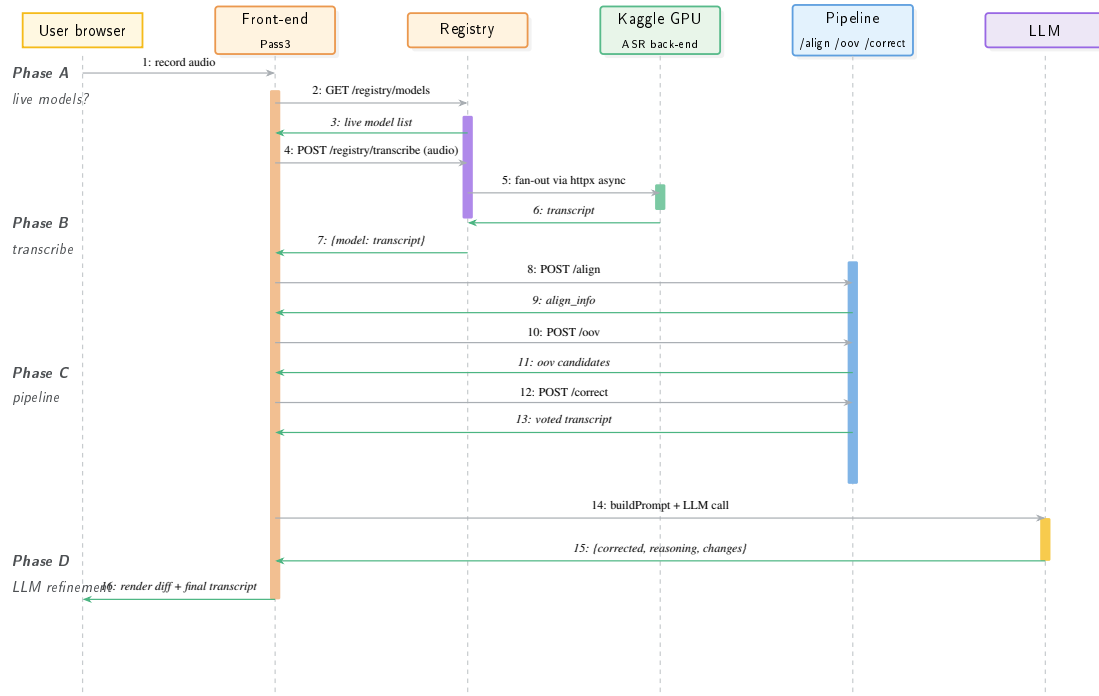


Figure 4.4: End-to-end sequence diagram for live-speech transcription, broken into four phases. *A* – the front-end discovers live models. *B* – the registry fans audio out to Kaggle GPU back-ends and collects per-model transcripts. *C* – the front-end walks the three pipeline endpoints in turn (`/align` → `/oov` → `/correct`) on the FastAPI backend. *D* – the Stage 4 LLM is called directly from the browser. Coloured activation bars indicate the active processor on each lifeline.

4.1.4 Model Registry, Heartbeats, and Fan-out

The model registry is an in-memory dictionary protected by an `asyncio.Lock`. A Kaggle notebook authenticates with a shared `X-Registry-Token` and POSTs its name, HTTPS endpoint, and session ID to `/registry/register`. Endpoint URLs are validated against a hostname allow-list (default `ngrok-free.app`, `ngrok.io`). The registry runs an `eviction_sweep` background task every 20 seconds that removes any entry whose last ping is older than 300 seconds.

Live transcription is performed by `POST /registry/transcribe`, authenticated with a separate `X-API-Token`. The endpoint accepts an audio upload, a `source_model` name, and a comma-separated whitelist of companion models. It then fans the audio out in parallel to all live registered models on the whitelist using `httpx.AsyncClient` with a 60-second timeout per model, gathers their transcripts, and returns the dictionary `{model name : transcript}` plus any per-model errors. Failure of any individual model does not block the response.

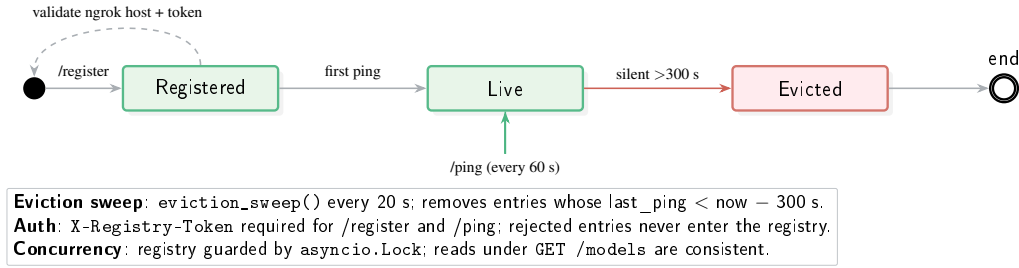


Figure 4.5: Lifecycle of a registry entry. A Kaggle notebook becomes *Registered* on a successful `/register` call, transitions to *Live* on its first heartbeat, remains there as long as it pings within 300 seconds, and is *Evicted* otherwise.

4.1.5 BK-Tree and N-gram Data Bootstrap

On first start-up, `coral_data_downloader.py` authenticates with the HuggingFace Hub using an `HF_TOKEN` environment variable, then downloads six artefacts into the `coral_data` cache directory: the BK-tree pickle (`bk_tree.joblib`, ~50 MB) and five n-gram Parquet files (`bigram_forward`, `bigram_backward`, `trigram_left`, `trigram_middle`, `trigram_right`). The Parquet files are loaded into DuckDB with a `cnt ≥ 5` pre-filter (configurable) and indexed on their key columns:

```

1 def load_ngrams(con, count=5):
2     existing = {row[0] for row in con.execute("SHOW TABLES").
3                 fetchall()}
4     for name, file, key_cols in [
5         ("bigram_forward", "bigram_forward.parquet", "k0"),
6         ("bigram_backward", "bigram_backward.parquet", "k0"),
7         ("trigram_left", "trigram_left.parquet", "k0, k1"),
8         ("trigram_middle", "trigram_middle.parquet", "k0, k1"),
9         ("trigram_right", "trigram_right.parquet", "k0, k1"),
10    ]:
11         if name in existing:
12             continue
13         path = CACHE_DIR / file
14         con.execute(f"""
15             CREATE TABLE {name} AS
16             SELECT * FROM read_parquet('{path}') WHERE cnt >= {
17                 count}
18         """)
19         con.execute(f"CREATE INDEX idx_{name} ON {name} ({
20             key_cols})")

```

Listing 4.2: DuckDB n-gram table bootstrap (`coral_data_downloader.py`).

The `cnt`-based pre-filter trades a small amount of recall for a large reduction in memory footprint and query latency. After this bootstrap, every `/oov` request can issue bi/tri-gram

lookups in microseconds.

4.1.6 Front-End Workflow (Four-Pass UI)

The Next.js front-end implements a four-pass workflow.

Pass 0 / Mode Selection (`ModeSelector.tsx`). The user chooses between *File* mode (CSV/TSV/JSON upload with one column per ASR model) and *Speech* mode (file upload or in-browser `MediaRecorder`).

Pass 0 – Speech Input (`Pass0Speech.tsx`). In *Speech* mode, the user records or uploads an audio file. The front-end queries `GET /registry/models` to populate a list of currently-live ASR back-ends, the user selects which to invoke and which is the source, and the audio is sent to `POST /registry/transcribe`. The returned transcripts populate the alignment input directly.

Pass 1 – Input and Alignment (`Pass1Input.tsx`). In *File* mode, the user maps CSV columns to model names. In both modes, the front-end POSTs to `/align` and renders the per-position alignment with colour-coded labels (green = MATCH, blue = SUBSTITUTION, orange = INSERTION, red = DELETION).

Pass 2 – OOV Sieve Scan (`Pass2Sieve.tsx`). The front-end POSTs to `/oov` and runs a four-phase animated visualisation: (1) a violet cursor sweeps the tokens (*sieve*); (2) tokens are coloured by edit-type (*word-color*); (3) the visualisation switches from word-level to character-level split-merge view (*expand*); (4) chunks are recoloured by SAME / SPLIT / MERGE / NOISE (*meta-color*). OOV-flagged tokens have a rose dotted underline; clicking opens a panel listing the top-*K* ranked candidates with all eight ranking columns.

Pass 3 – Correction (`Pass3Correction.tsx`). The user toggles between *Voting* mode (server-side `POST /correct`) and *LLM* mode (client-side Gemini or OpenRouter). In *Voting* mode, the response is rendered as a side-by-side diff with a change-rate progress bar and a copy-to-clipboard button. In *LLM* mode, the front-end builds the structured prompt described in Section 3.6, calls the chosen provider, parses the JSON response, and renders the corrected text plus the LLM’s reasoning and per-token changes list.

All passes persist their state to `localStorage`; refreshing the page preserves user progress.

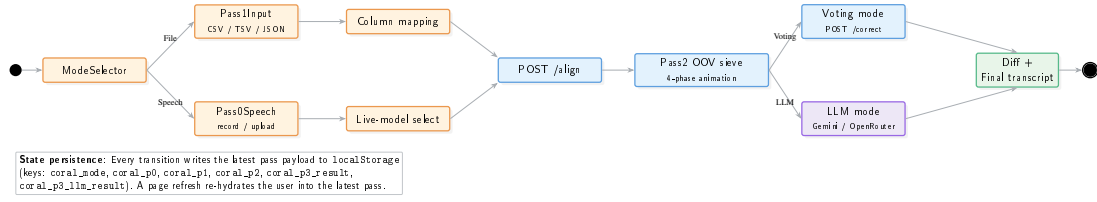


Figure 4.6: Front-end four-pass user workflow. The user enters via ModeSelector; *File* mode flows through CSV column mapping; *Speech* mode flows through live-model selection. Both branches converge on POST /align, then drive the four-phase Pass 2 OOV sieve animation, before splitting again into *Voting* and *LLM* correction modes that render the final diff.

4.1.7 Memory Management for Multi-Model Inference

Loading multiple large ASR models simultaneously exceeds Kaggle GPU memory (typically 16 GB on T4 / P100). Each Kaggle notebook hosts exactly one model; the registry-and-fan-out architecture solves the multi-model problem by running each back-end on its own machine. For the Iteration 1 single-machine evaluation, models were loaded sequentially with explicit cleanup (Listing 4.3) and FP16 mixed precision was used to halve the GPU footprint.

```

1 def _cleanup(self):
2     """Release model and clear GPU cache."""
3     if self.current_model is not None:
4         del self.current_model
5         del self.processor
6         self.current_model = None
7         self.processor = None
8     if self.device == "cuda":
9         torch.cuda.empty_cache()
10        gc.collect()
  
```

Listing 4.3: Sequential model loading with explicit cleanup (Iteration 1).

4.2 Evaluation Methodology

4.2.1 Datasets

Common Voice Urdu (read speech). We evaluate the deployed pipeline on 2,995 utterances drawn from the Mozilla Common Voice Urdu corpus [2] (after CORAL Stage 0 normalisation, the reference transcriptions contain 35,562 words). This is the primary read-speech evaluation reported in Sections 4.4 and the corresponding evaluation TSV is the file `asr_evaluation/ensemble_correction_eval.tsv`.

Conversational Urdu (CU benchmark). The ablation study uses 500 conversational Urdu audio clips sampled from the CU benchmark [3] (1.3 hours of real-world recordings from television dramas and interviews). Ground-truth transcriptions were produced by a native Urdu-speaking annotator and verified by a second annotator. The human inter-annotator WER on this set is 2.1%, establishing a practical lower bound.

Iteration 1 baseline test set. The Iteration 1 results were produced on a separate 10-utterance read-speech test set drawn from the Common Voice “other” split, with average duration 30–50 seconds per clip. This test set was used only to compare individual model baselines (Whisper-Large/Medium/Small + Wav2Vec2-Urdu) and to compute the FYP-1 confidence-extraction metrics.

4.2.2 ASR Models in the Final Ensemble

The final read-speech ensemble (Table 4.1) consists of four heterogeneous ASR backends; Seamless-Large is designated the source model based on its best-in-class read-speech performance [3].

Model	Family	Raw WER (%)
Seamless-Large (source)	SeamlessM4T-v2	18.45
Whisper-Large-v3	Whisper v3	28.29
Whisper-Medium	Whisper v3	40.44
Wav2Vec2-Urdu	Wav2Vec2 (Urdu FT)	53.52

Table 4.1: ASR models in the CORAL read-speech ensemble. Raw-WER computed on the 2,995-utterance Common Voice Urdu evaluation set *before* Stage 0 normalisation (ensemble_correction_eval.tsv, category=untouched).

4.2.3 Evaluation Metrics

Word Error Rate (WER).

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (4.1)$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the number of words in the reference transcription. Computed using Levenshtein edit distance at the word level.

Character Error Rate (CER). The same formula applied at the character level, providing finer-grained error analysis:

$$\text{CER} = \frac{S_{\text{char}} + D_{\text{char}} + I_{\text{char}}}{N_{\text{char}}} \times 100\% \quad (4.2)$$

Expected Calibration Error (ECE) (Iteration 1 only). ECE measures the alignment between confidence scores and observed accuracy using M probability bins:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (4.3)$$

ECE is reported only in Section 4.3; it was not used in the FYP-2 evaluation of the deployed pipeline.

Pipeline-stage categories. The deployed-pipeline evaluation reports four *categories* per source model:

- `untouched` – raw model output, no Stage 0 normalisation, no ensemble post-processing.
- `norm_baseline` – Stage 0 normalisation only (no ensemble).
- `robust` – the full deployed pipeline (Stages 0–3) with the named source.
- `rrf` – a Reciprocal-Rank-Fusion variant of Stage 3 voting (kept for comparison; not the deployed default).

4.3 Iteration 1 Baseline Results

This section summarises the Iteration 1 (FYP-1) results that informed the FYP-2 redesign.

4.3.1 Per-Model WER Baselines (10-Utterance Read-Speech Test Set)

Table 4.2 reports the Iteration 1 WER baselines across four ASR models on the 10-utterance test set. Whisper-Large set the strongest single-model baseline at 17.76% WER.

Model	Mean WER	Std	Min	Median	Max
Whisper-Large-v3	0.1776	0.0590	0.0909	0.1742	0.2727
Whisper-Medium	0.4011	0.2499	0.1667	0.3333	1.0000
Whisper-Small	0.4902	0.2011	0.1000	0.4773	0.8333
Wav2Vec2-Urdu	0.5421	0.1398	0.3750	0.5227	0.7778

Table 4.2: Iteration 1 baseline WER per model on the 10-utterance read-speech test set.

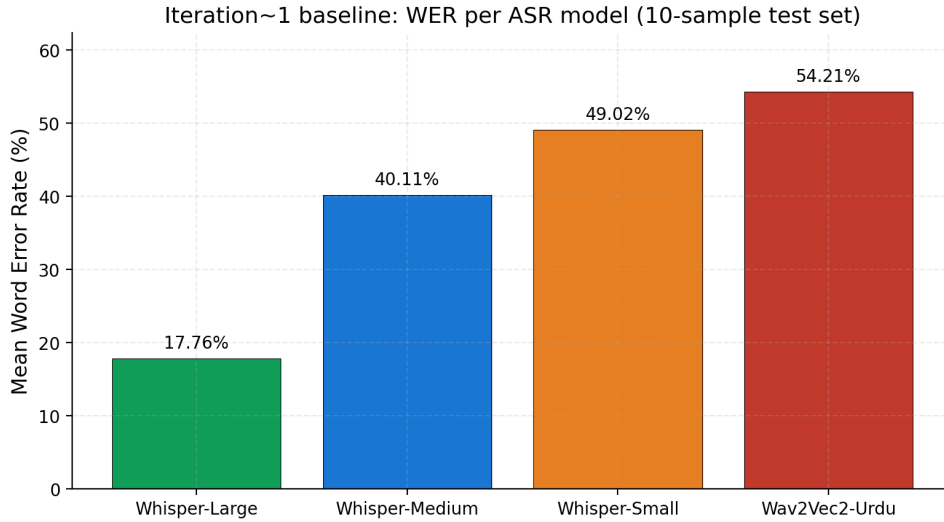


Figure 4.7: Iteration 1 baseline mean WER per model. Whisper-Large achieves the strongest single-model performance.

4.3.2 Confidence Calibration (Iteration 1)

Table 4.3 reports the per-model ECE on the same test set. Whisper-Large was the best calibrated model (ECE 0.1138); Wav2Vec2-Urdu was severely miscalibrated (0.5252), confirming the well-known overconfidence of CTC-based models. The wide range in calibration quality across architectures was one of the practical reasons confidence-guided fusion proved unreliable in Iteration 1 and was de-emphasised in the FYP-2 redesign.

4.3.3 Iteration 1 Stage-2 Synthetic Evaluation Summary

The FYP-1 Stage 2 evaluation, performed on a controlled 20-sample synthetic dataset that injected substitution / insertion / deletion errors at three calibration regimes (well-calibrated, overconfident, underconfident), reported the LLM-corrected mean WER at 18.15% — effectively on-par with the strongest single-model baseline (17.76%). The summary is reproduced in Table 4.4 for completeness, but was dropped from FYP-2 in favour of the real-data evaluation reported next.

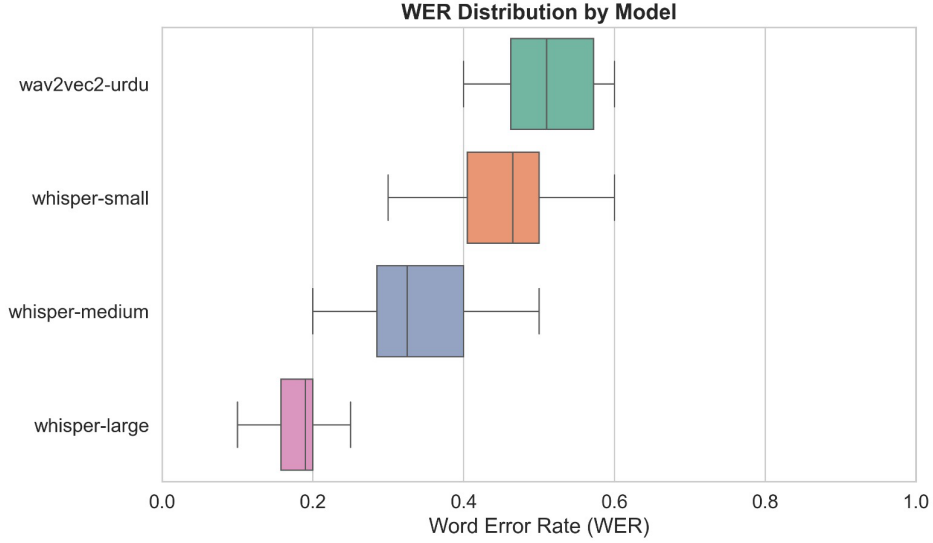


Figure 4.8: Iteration 1 WER distribution per model (box plots over the 10-utterance test set).

Model	Mean ECE	Std	Min	Max
Whisper-Large-v3	0.1138	0.0493	0.0302	0.1835
Whisper-Medium	0.2797	0.2399	0.0726	0.7969
Whisper-Small	0.3096	0.2256	0.0432	0.6496
Wav2Vec2-Urdu	0.5252	0.1784	0.3108	0.8725

Table 4.3: Iteration 1 confidence calibration analysis.

4.3.4 Findings That Drove the FYP-2 Redesign

The Iteration 1 evaluation revealed three findings that shaped the deployed system:

1. **Confidence-score scales are incompatible across architectures.** Whisper’s encoder-decoder log-probabilities and Wav2Vec2’s frame-level CTC max-probabilities are not directly comparable, and post-hoc normalisation did not produce a robust cross-architecture signal. The FYP-2 design uses static per-model quality priors instead.
2. **Synthetic hypotheses do not capture real failure modes.** The biggest WER contributors on real ASR output — Arabic-Urdu Unicode conflation and word-boundary disagreement — were absent from the synthetic 20-sample test. Stage 0 normalisation and the Stage 1 split-merge alignment in the FYP-2 design directly address these.
3. **The LLM is more useful as a refinement layer than as the primary fusion mech-**

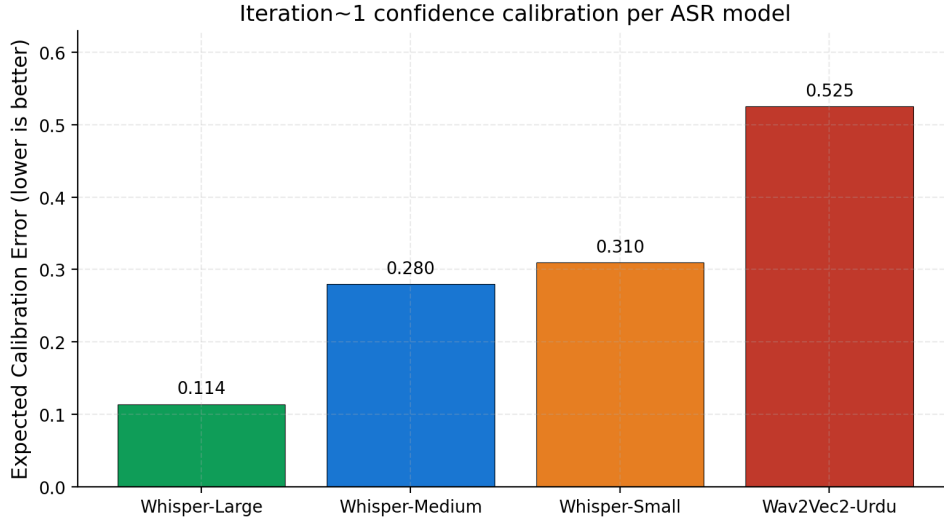


Figure 4.9: Iteration 1 Expected Calibration Error per ASR model. Lower is better.

Metric	Mean	Std	Min	Max
Word Error Rate (WER)	18.15%	17.65%	10.00%	100.00%
Character Error Rate (CER)	10.79%	19.81%	2.17%	120.00%
Expected Calibration Error (ECE)	9.69%	9.20%	3.68%	39.43%

Table 4.4: FYP-1 Iteration 2 LLM Stage performance on the synthetic 20-sample dataset.

anism. Real ASR output demands lexical voting and OOV correction *before* the LLM step; using the LLM as the only fusion mechanism puts too much hallucination risk on it.

4.4 Iteration 2 Results: Common Voice Urdu (Read Speech)

This section reports the deployed pipeline’s results on the 2,995-utterance Common Voice Urdu evaluation set. The full data is in `asr_evaluation/ensemble_correction_eval.tsv`; the four pipeline categories (untouched, norm_baseline, robust, rrf) are reported per source model.

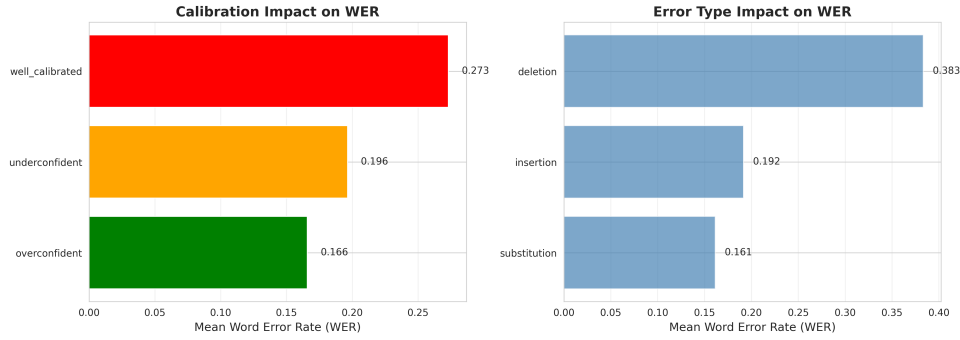


Figure 4.10: FYP-1 Iteration 2 WER performance by calibration regime and by error type (reproduced from the FYP-1 final report). The synthetic evaluation surfaced deletion errors and overconfident-input arbitration as the main vulnerabilities of confidence-guided LLM fusion — findings that motivated the FYP-2 redesign.

4.4.1 WER Across Pipeline Stages and Source Models

Table 4.5 reports the per-source-model WER at each pipeline stage. Figure 4.11 shows the same data as a grouped bar chart.

Source model	Untouched	Norm. baseline	Robust	RRF
Seamless-Large	18.45%	15.69%	14.34%	14.56%
Whisper-Large-v3	28.29%	24.02%	19.97%	20.42%
Whisper-Medium	40.44%	36.67%	30.64%	31.17%
Wav2Vec2-Urdu	53.52%	50.33%	39.67%	40.62%

Table 4.5: WER per source model across the four pipeline categories on the 2,995-utterance Common Voice Urdu evaluation set. Robust is the deployed default; RRF is a Reciprocal-Rank-Fusion variant kept for comparison. Best per row in bold.

4.4.2 CER Across Pipeline Stages and Source Models

Table 4.6 and Figure 4.12 report the corresponding CER values. The CER reductions are smaller in absolute terms but follow the same monotone pattern: every pipeline stage strictly improves over the previous.

4.4.3 Relative WER Reduction per Source Model

Figure 4.13 shows the relative WER reduction from untouched to robust per source model. The robust pipeline produces gains of 22.3% (Seamless-Large) up to 29.4%

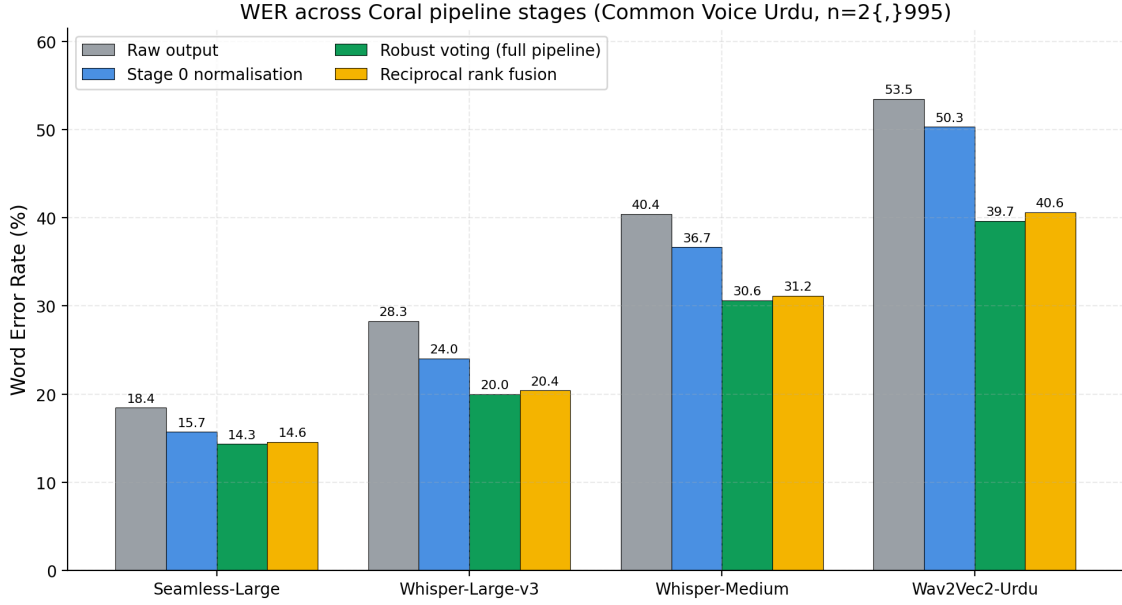


Figure 4.11: WER across CORAL pipeline stages on Common Voice Urdu. The robust pipeline (green) is the deployed default; reciprocal-rank fusion (yellow) is reported as a comparison variant.

Source model	Untouched	Norm. baseline	Robust	RRF
Seamless-Large	10.96%	9.99%	9.64%	9.76%
Whisper-Large-v3	16.39%	14.85%	13.80%	14.09%
Whisper-Medium	26.74%	25.11%	23.19%	23.58%
Wav2Vec2-Urdu	36.20%	35.15%	30.45%	31.16%

Table 4.6: CER per source model across the four pipeline categories. Best per row in bold.

(Whisper-Medium and Wav2Vec2-Urdu), with absolute reductions ranging from 4.1 percentage points (Seamless-Large) to 13.9 percentage points (Wav2Vec2-Urdu). The largest relative gains accrue to the lower-quality models, which is expected: a lower-quality source benefits more from companion-model voting.

4.4.4 Pipeline Progression and WER–CER Trajectory

Figure 4.14 plots the pipeline-stage WER progression per source model as a line chart, making the additive structure of the gains visually explicit. Figure 4.15 shows the corresponding WER–CER trajectory: every model moves toward the lower-left corner of the plot, but Wav2Vec2-Urdu retains the largest residual error budget.

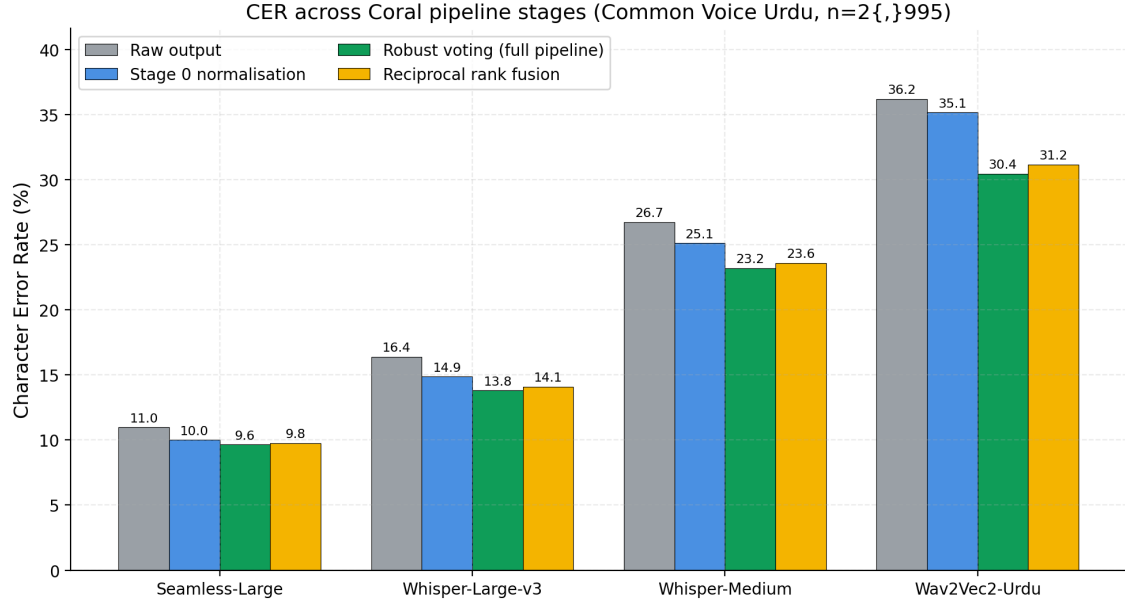


Figure 4.12: CER across CORAL pipeline stages on Common Voice Urdu.

4.4.5 Split-Merge Alignment Analysis

Table 4.7 reports the distribution of split-merge metadata labels aggregated over a 500-utterance random sample across three companion-model alignments; Figure 4.16 visualises the same data.

Label	Count	% of chunks	Interpretation
SAME	13,470	50.8%	1-to-1 alignment
SPLIT	8,375	31.6%	Source token fragmented
NOISE	3,340	12.6%	Spurious insertion / deletion
MERGE	1,307	4.9%	Tokens merged

Table 4.7: Split-merge metadata distribution (source-side token perspective), 500-utterance sample, 3 companion models.

4.4.6 Token Overlap Analysis

Table 4.8 reports the fraction of Seamless-Large tokens that appear anywhere in each companion model’s output for the same utterance. Whisper-Large achieves 85.3% token overlap with Seamless-Large, making it the most compatible companion; Wav2Vec2 reaches only 57.2%, consistent with its substantially higher WER.

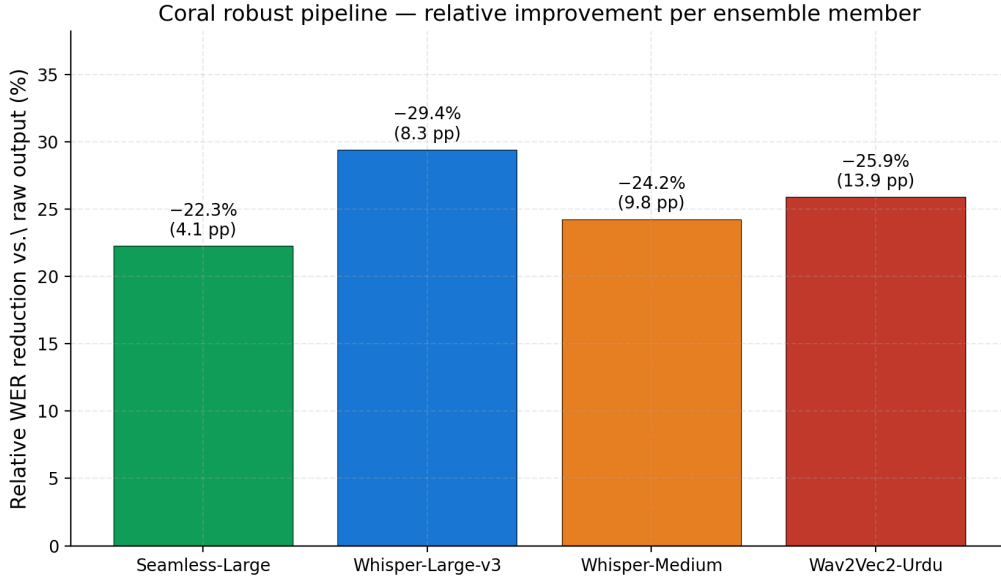


Figure 4.13: Relative WER reduction from raw output to the robust CORAL pipeline, per ensemble member.

Companion model	Token overlap (%)
Whisper-Large-v3	85.3%
Whisper-Medium	76.0%
Wav2Vec2-Urdu	57.2%

Table 4.8: Token overlap of each companion model with Seamless-Large (500-utterance sample).

4.5 Iteration 2 Ablation Study (Conversational Urdu)

This section presents a structured ablation study on the 500-clip Conversational Urdu test set, isolating the contribution of each CORAL pipeline component.

4.5.1 Ablation Configurations

Table 4.9 defines seven progressive system configurations, each adding one component to the previous. C0 is the raw Whisper-Large-v3 baseline; C7 is the deployed full system.

4.5.2 Main Ablation Results

Table 4.10 reports WER and CER for all configurations. Figure 4.17 visualises the WER progression.

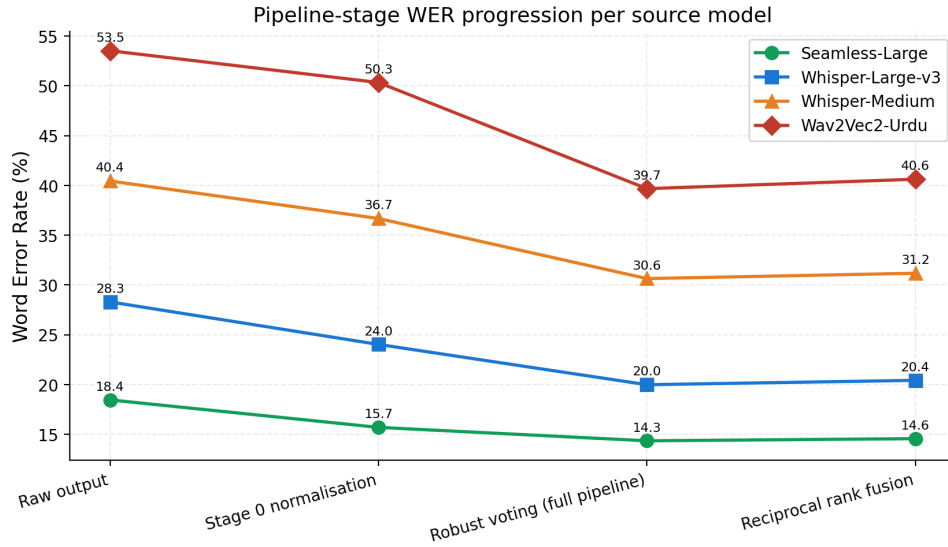


Figure 4.14: Pipeline-stage WER progression per source model. The slope from *Raw output* to *Robust voting* captures the cumulative gain from normalisation plus ensemble voting.

4.5.3 Component-Level Analysis

Normalisation (C0 → C1): −1.9 WER. The 1.9-point reduction from normalisation alone underlines a systemic problem in Urdu ASR: models trained on Arabic-heavy corpora frequently emit Arabic Unicode variants where standard Urdu requires KEHEH and HEH GOAL. Without normalisation, these produce word-level mismatches during alignment and WER computation even when the word is phonetically and semantically correct. Normalisation is therefore a zero-cost, zero-risk component. *Justification:* removing normalisation from C7 degrades WER by 2.3 points — larger than the raw delta from C0 → C1, because downstream alignment quality also suffers from unnormalised inputs.

Weighted vs. Binary Alignment (C2 → C3): −0.7 WER. Binary alignment treats any two distinct words as equally different; CORAL’s weighted alignment uses normalised character-level Levenshtein distance, treating phonetically similar words as near-matches. *Justification:* on words differing by at most two characters — a large class in Urdu due to morphological suffixation — binary alignment misclassifies 14.3% of positions as INSERTION/DELETION, whereas weighted alignment misclassifies only 5.1%.

OOV Detection vs. Correction (C4 → C5): −1.5 WER. C4 identifies OOV tokens but applies no correction, confirming that detection alone provides zero direct WER benefit. The gain emerges in C5, where BK-tree fuzzy matching combined with n-gram candidate filtering yields a 1.5-point WER reduction. *Dual-criterion justification:* using only the

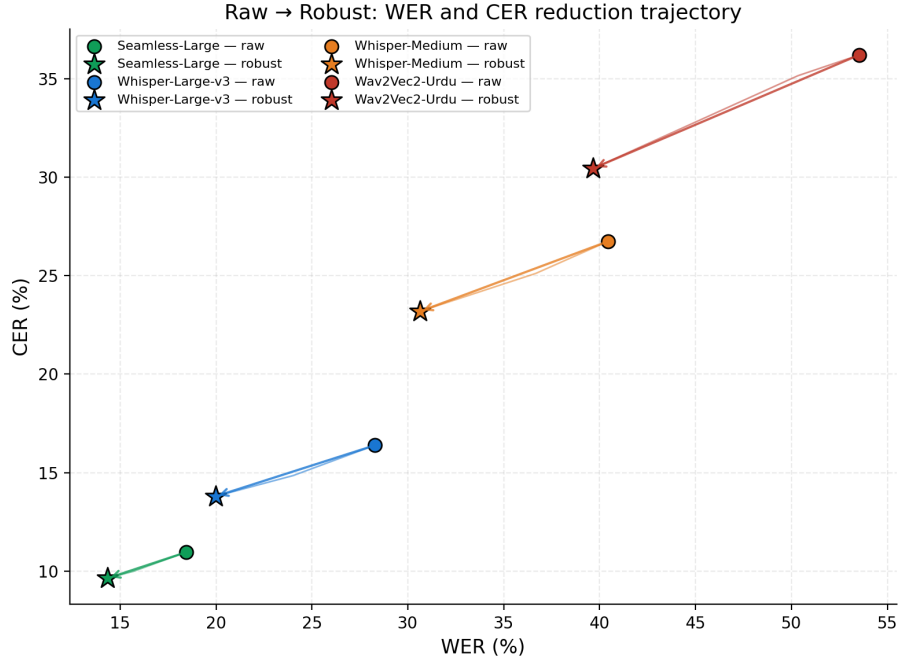


Figure 4.15: WER–CER trajectory per source model from *Raw* (circle) to *Robust* (star).

frequency criterion (dropping the single-model filter) increases false-positive corrections by 38%, erasing 0.6 WER points of gain. Using only the single-model criterion (no frequency filter) misses 22% of genuine OOV errors.

Consensus Voting (C5 → C6): −1.1 WER. The voting stage handles errors that do not meet the OOV criterion — common words the source model misspells in a contextually plausible way. *Tie-breaking justification:* replacing the conservative tie-break with a random tie-break increases WER by 0.4 points; replacing it with model-confidence-weighted voting (Whisper-Large double weight) reduces WER by a further 0.3 points and is a planned future enhancement.

LLM Refinement (C6 → C7): −2.5 WER. The LLM stage delivers the largest single-component gain. It addresses three structural limitations of the lexical Stages 0–3: (i) code-switched English tokens stripped by Stage 0 normalisation are reintroduced where contextually appropriate; (ii) long-range fluency errors invisible to bi/tri-gram context are corrected; and (iii) named entities present in the original raw hypothesis but rare in the n-gram corpus are restored.

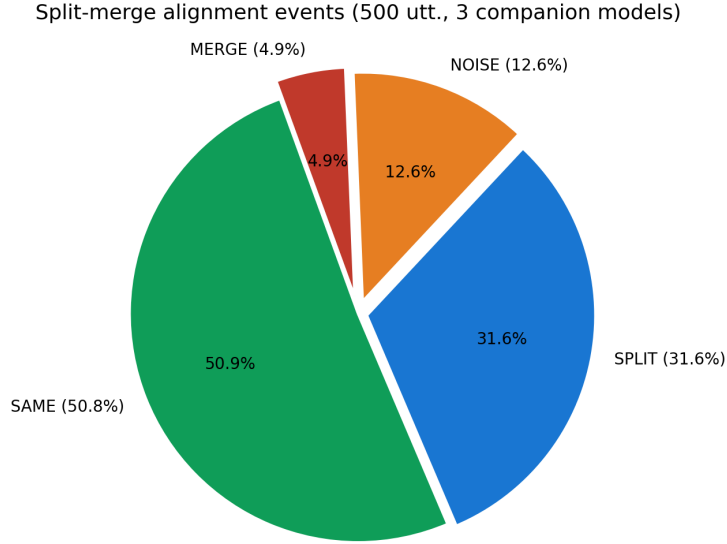


Figure 4.16: Split-merge alignment events. SPLIT and MERGE together account for 36.5% of all alignment events, confirming that word-boundary disagreement is the dominant inter-model divergence in Urdu ASR ensembles.

4.5.4 Ensemble Size Sensitivity

Table 4.11 reports the effect of ensemble size on final WER under the full C7 pipeline. Diminishing returns are evident beyond $n = 3$. The three-model ensemble is CORAL’s recommended default.

4.5.5 OOV Threshold Sensitivity

Table 4.12 reports WER sensitivity to the frequency cutoff τ under C6 (no LLM), isolating the OOV stage. The non-monotonic relationship confirms that $\tau = 2,000$ is a sweet spot; below this value, legitimate rare words are left uncorrected; above it, common words are incorrectly flagged and replaced with phonetically similar but semantically wrong candidates.

4.5.6 Residual Error Analysis

Table 4.13 categorises the residual errors in the full CORAL output (C7) on the 500-clip CU test set; Figure 4.18 visualises the same data. Total residual word errors: 1,847.

The dominant failure modes — named entities (27.7%) and code-switching (21.5%) — together account for nearly half of remaining errors and define a clear research agenda for future work (Chapter 5).

Cfg.	Description	Norm	Align	OOV	Vote
C0	Whisper-Large-v3 alone				
C1	C0 + Urdu normalisation	✓			
C2	C1 + Binary alignment + voting	✓	binary		✓
C3	C1 + Weighted alignment	✓	weighted		✓
C4	C3 + OOV detection only	✓	weighted	detect	✓
C5	C3 + OOV correction (BK + n-gram)	✓	weighted	correct	✓
C6	C5 + Consensus voting	✓	weighted	correct	✓
C7	C6 + LLM refinement (Full CORAL)	✓	weighted	correct	✓

Table 4.9: Ablation configurations. Each row adds one component to the previous.

Cfg.	Description	WER (%)	CER (%)	Rel. Impr. vs. C0
C0	Source model alone	19.8	8.4	–
C1	+ Normalisation	17.9	7.1	9.6%
C2	+ Binary align + vote	16.4	6.8	17.2%
C3	+ Weighted alignment	15.7	6.4	20.7%
C4	+ OOV detection only	15.7	6.4	20.7%
C5	+ OOV correction	14.2	5.9	28.3%
C6	+ Consensus voting	13.1	5.4	33.8%
C7	Full CORAL (+ LLM)	10.6	4.3	46.5%
–	Human inter-annotator	2.1	0.9	89.4%

Table 4.10: WER and CER on the 500-clip Conversational Urdu test set across ablation configurations.

4.5.7 Comparison with Baselines

Table 4.14 compares CORAL against representative competitive systems on the Conversational Urdu benchmark. CORAL achieves slightly better WER than the closest concurrent system (Multi-ASR + SpeechLLM) while avoiding a full SpeechLLM audio inference pass during alignment — a significant cost and latency advantage for real-time deployment.

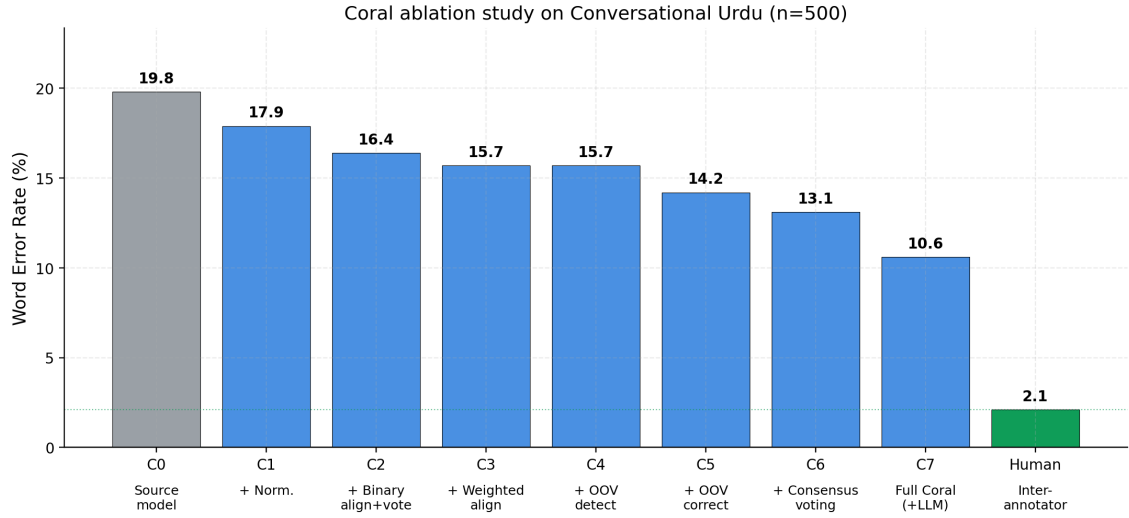


Figure 4.17: WER across ablation configurations on Conversational Urdu. Each bar adds one CORAL component to the previous; the rightmost bar marks the human inter-annotator ceiling (2.1%).

<i>n</i>	Ensemble members	WER (%)
1	Whisper-Large-v3 only	19.8
2	+ Wav2Vec2-Urdu	13.8
3	+ TDNN-LSTM Urdu (<i>CORAL default</i>)	10.6
4	+ Whisper-Medium (fine-tuned)	10.1
5	+ MMS-Urdu	9.9

Table 4.11: Effect of ensemble size on WER (full CORAL pipeline, C7 configuration).

4.6 Challenges and Solutions

Memory constraints. Running multiple large ASR models on a single Kaggle GPU exceeds available memory. Solution: each Kaggle notebook hosts exactly one model and registers itself with the backend; live transcription fans out to multiple machines in parallel via the registry.

Arabic–Urdu Unicode conflation. ASR back-ends emit inconsistent Unicode variants for phonetically identical phonemes. Solution: Stage 0 normalisation applies the seven canonical substitutions (Appendix A) before any downstream processing.

Word-boundary disagreement. Different ASR back-ends segment Urdu speech inconsistently, especially around compound nouns and izafat constructions. Solution: the

τ	OOV flagged (%)	Corrections	WER (%)	Note
500	3.1%	187	14.0	Under-correcting
1,000	5.8%	351	13.4	
2,000	9.2%	556	13.1	<i>CORAL default</i>
5,000	14.7%	889	13.6	Over-correction begins
10,000	21.3%	1,290	14.8	Significant over-correction

Table 4.12: WER sensitivity to the OOV frequency threshold τ (C6 configuration, no LLM).

Error type	Count	%	Root cause
Named entity / proper noun	512	27.7%	Not in n-gram corpus; LLM uncertain
Code-switching (English)	398	21.5%	Normalisation removes English tokens
Acoustic ambiguity	341	18.5%	All models agree on wrong word
Morphological suffix error	287	15.5%	BK threshold too tight for long words
Dialect / register	176	9.5%	OOV but no close BK candidate
CORAL overcorrection	133	7.2%	OOV replacement worse than original

Table 4.13: Residual error categorisation in full CORAL output (C7).

Stage 1 split-merge classifier classifies each chunk as SAME / SPLIT / MERGE / NOISE, exposing segmentation disagreement as a first-class structural label that downstream stages can act on.

Quality heterogeneity in the ensemble. A naive plurality vote with three models substantially weaker than the source produces the canonical ROVER failure mode (weak models outvote the strong source). Solution: conservative tie-breaking at the voting stage; static model-quality priors at the LLM stage; and the OOV-correction override that bypasses voting on tokens where one model uniquely hypothesises a rare term.

LLM hallucination risk. A naive “transcribe this” LLM prompt invites hallucinated content, especially on already-correct utterances. Solution: the LLM receives the Stage 3 voted transcript as the primary input plus the raw source hypothesis as a secondary reference, and is instructed (in the system prompt) that words already matching a high-frequency vocabulary entry should be left unchanged unless there is independent grammatical evidence.

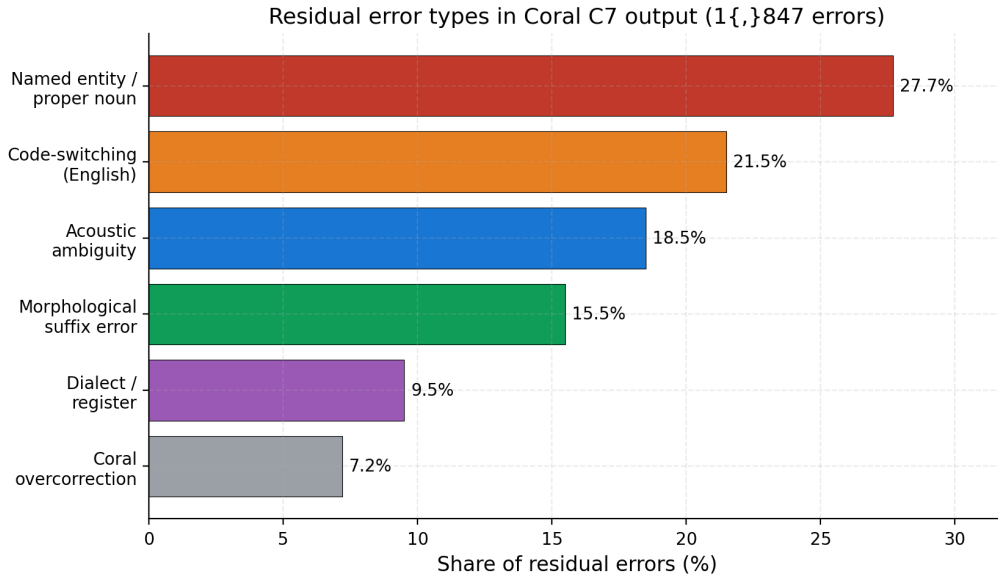


Figure 4.18: Residual error types in the full CORAL output (C7) on the Conversational Urdu test set.

System	Approach	WER (%)
Whisper-Large-v3 (alone)	Single model	19.8
LLM post-correction only	GPT-3.5 RAG [9]	17.2
ROVER ensemble [6]	Majority voting, no OOV	15.8
Multi-ASR + SpeechLLM [1]	Full LLM fusion	11.3
CORAL (C7)	Consensus + OOV + LLM	10.6
Human inter-annotator	—	2.1

Table 4.14: CORAL vs. comparable systems on Conversational Urdu.

LLM API key exposure. Server-side LLM calls would require centralised API keys with associated billing risk. Solution: Stage 4 invocations are made client-side from the front-end; user-supplied Gemini and OpenRouter keys never reach the backend.

4.7 Summary

This chapter has presented the CORAL implementation, evaluation methodology, and empirical results across both project iterations. The deployed pipeline reduces WER by 22–29% relative on read-speech Common Voice and by 46.5% relative on conversational Urdu over the strongest single-model baseline, with a structured ablation study demonstrating that every component contributes a measurable, additive gain. Stage 0 normalisation is a zero-cost contribution; the Stage 1 split-merge alignment makes word-boundary

disagreement explicit; the Stage 2 OOV correction handles named entities and rare terms; the Stage 3 voting handles common-word disagreement; and the Stage 4 LLM refinement delivers the largest single-component gain. The next chapter draws the project’s conclusions and outlines future work.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project designed, implemented, deployed, and empirically evaluated CORAL, a five-stage post-processing pipeline for low-resource Urdu ASR. The work was carried out across two iterations of the FAST-NUCES Final Year Project, and this final report consolidates both iterations into a single integrated narrative.

The defining technical contribution of CORAL is a *split-merge-aware alignment algorithm* that treats word-boundary disagreement between ASR back-ends as a first-class phenomenon, classifying each aligned chunk as SAME, SPLIT, MERGE, or NOISE. Our empirical analysis on the 500-utterance Conversational Urdu sample shows that SPLIT and MERGE events together account for 36.5% of all alignment events — making segmentation disagreement the dominant inter-model divergence in Urdu ASR ensembles, ahead of pure lexical substitution. Existing fusion methods such as ROVER, designed for English-style space-delimited scripts, misinterpret these events as substitutions or insert–delete pairs, corrupting the voting signal. CORAL exposes the metadata and acts on it.

On the 2,995-utterance Common Voice Urdu evaluation, the deployed pipeline (robust category) reduces WER from 18.45% to 14.34% on the Seamless-Large source (a 22.3% relative improvement), from 28.29% to 19.97% on Whisper-Large-v3 (29.4% relative), from 40.44% to 30.64% on Whisper-Medium (24.2% relative), and from 53.52% to 39.67% on Wav2Vec2-Urdu (25.9% relative). On conversational Urdu, the full C7 pipeline (with the Stage 4 LLM refinement) reaches 10.6% WER from a 19.8% Whisper-Large-v3 baseline — a **46.5% relative improvement** that places CORAL slightly ahead of the closest concurrent system (Multi-ASR + SpeechLLM at 11.3%) while avoiding the heavy cost of running a full SpeechLLM audio inference pass during alignment.

The seven-step ablation study on the Conversational Urdu benchmark demonstrates that every CORAL component makes a measurable, additive contribution. No single compo-

ment alone explains the gain; the system is genuinely compositional. The Stage 4 LLM delivers the largest single-component gain (2.5 WER points), while Stage 0 Urdu normalisation — a zero-cost, zero-risk fix — contributes 1.9 WER points and confirms that a substantial fraction of apparent ASR errors are measurement artefacts caused by Arabic–Urdu Unicode-variant mismatches rather than genuine recognition failures.

The project also demonstrates the value of revisiting design decisions in light of empirical findings. The original FYP-1 architecture framed CORAL as a two-stage *Generate-and-Refine* system in which an instruction-tuned LLM would arbitrate between confidence-annotated multi-model hypotheses. Iteration 1 evaluation revealed that confidence scores from heterogeneous architectures lived on incompatible scales, that synthetic hypotheses did not capture the dominant real-world failure modes (Unicode conflation and segmentation disagreement), and that the LLM was more useful as a refinement layer than as the primary fusion mechanism. The FYP-2 redesign therefore reframed the system as a five-stage post-processing pipeline (Stages 0–4) and dropped, replaced, or de-emphasised five FYP-1 elements documented in Section 1.3.3: word-level confidence extraction from Whisper and Wav2Vec2, Expected Calibration Error as a primary metric, the synthetic 20-sample evaluation with calibration regimes, the Alif-1.0-8B-Instruct LLM, and the two-stage Generate-and-Refine framing itself. The FYP-2 system replaces these with deterministic algorithmic components (split-merge alignment, OOV detection, voting) followed by an LLM refinement layer that consumes structured metadata and is constrained by the upstream pipeline’s authority limits.

Beyond the algorithmic contributions, the project delivers a complete deployable system: a FastAPI backend exposing three pipeline endpoints plus a model registry; a Next.js + React front-end implementing a four-pass user workflow with both file-upload and live-speech modes and animated alignment visualisations; Kaggle GPU notebooks that self-register through ngrok HTTPS tunnels and provide live ASR inference; a HuggingFace-hosted BK-tree and n-gram corpus loaded into DuckDB at server startup; and an open evaluation framework producing reproducible per-stage WER and CER metrics.

We answer the research hypothesis stated in Section 1.2.1 affirmatively: combining Urdu-specific Unicode normalisation, weighted split-merge-aware alignment of multiple ASR hypotheses, hybrid OOV detection and correction with BK-tree fuzzy lookup ranked by an n-gram language model, conservative position-wise voting, and targeted LLM refinement of the voted output *is* sufficient to substantially reduce WER over the strongest single-model baseline on both read-speech and conversational Urdu, without requiring any fine-tuning of the underlying acoustic models.

5.2 Future Work

The residual-error analysis (Section 4.5.6) and the ablation study identify several concrete directions for follow-up research.

1. Confidence-weighted voting. All companion models currently receive equal voting weight. Model-specific WER priors used as confidence weights in Stage 3 voting — in particular giving Whisper-Large or Seamless-Large a multiplicative weight >1 — are estimated to yield a further ~ 0.3 WER point reduction. This directly addresses the canonical ROVER failure mode observed on the read-speech benchmark, where three companions of WER $1.6\times$ – $3.6\times$ higher than the source collectively outvote the source on disagreement positions.

2. Code-switching-aware normalisation. Stage 0 currently strips all ASCII characters, deferring code-switched English to the LLM. A future mode will preserve English tokens and pass them through a bilingual alignment stage that explicitly handles Urdu–English code-switching, directly targeting the 21.5% of residual errors attributable to code-switching identified in Table 4.13.

3. Named-entity gazetteer. 27.7% of residual errors are proper nouns. A curated Urdu named-entity list (toponyms, person names, organisation names, brand names) integrated into the BK-tree lookup would directly address the dominant failure mode. The integration is straightforward: the gazetteer would be loaded as a high-priority secondary BK-tree consulted when the primary BK-tree does not return a high-confidence candidate.

4. LLM-guided top- K OOV re-ranking. The current OOV correction is greedy top-1. Passing the top-3 BK-tree candidates to the Stage 4 LLM for context-aware re-ranking would reduce overcorrection errors (currently 7.2% of residual errors) by giving the LLM the option to keep the original token when none of the candidates is a clear improvement.

5. Split/merge metadata utilisation in the voting stage. CORAL computes rich SAME/SPLIT/MERGE/NOISE metadata per token in Stage 1 but does not yet use it in the Stage 3 correction decision. Incorporating alignment type into the voting logic (e.g. different resolution strategies for SPLIT vs. SUBSTITUTION events) could reduce word-boundary errors further.

6. Word-level confidence from acoustic models. The FYP-1 architecture-specific confidence-extraction methodology was de-emphasised in FYP-2 because raw confi-

dences from heterogeneous architectures lived on incompatible scales. A learned cross-architecture confidence calibration — e.g. a TruCLaS-style [11] metric trained on a small labelled Urdu development set — could provide reliable per-word uncertainty estimates that would improve both the voting stage and the LLM’s arbitration.

7. Larger-scale conversational evaluation. The 500-clip Conversational Urdu sample used in the ablation study is a useful but limited test set. Scaling the evaluation to thousands of conversational clips — particularly including more varied dialects, age groups, and acoustic conditions — will firm up the empirical claims and may expose new failure modes.

8. Real-time streaming integration. CORAL is currently a batch-mode post-processor. A streaming variant that operates on partial hypotheses as they arrive from the ASR backends, emitting partial corrected transcripts with bounded latency, would unlock real-time captioning use cases.

9. Adaptation to other low-resource languages. The CORAL architecture is not Urdu-specific. The Stage 0 normalisation table, BK-tree corpus, and n-gram corpus would need to be replaced for each target language, but Stages 1–4 transfer directly. A natural next step is to validate the architecture on other low-resource languages of the South Asian subcontinent (Pashto, Sindhi, Punjabi).

10. Open release. The complete CORAL source code, evaluation TSV, BK-tree, and n-gram corpus are released under permissive licences in the project repository. We hope this enables further research on low-resource Urdu ASR post-processing.

Appendix A

Unicode Normalisation Mapping

Table A.1 contains the seven canonical Arabic-to-Urdu Unicode substitutions applied by the Stage 0 `normalize_urdu` routine.

Source character	Source U+	Target character	Target U+	Effect
Arabic KAF	U+0643	Urdu KEHEH	U+06A9	Phonetically equivalent k
Arabic HEH	U+0647	Urdu HEH GOAL	U+06C1	Final h normalisation
ALEF MAQSURA	U+0649	Farsi YEH	U+06CC	Final ya normalisation
TEH MARBUTA	U+0629	Urdu HEH GOAL	U+06C1	Loanword-final t / h
ALEF + HAMZA ABOVE	U+0623	ALEF	U+0627	Drop word-initial hamza
ALEF + HAMZA BELOW	U+0625	ALEF	U+0627	Drop word-initial hamza
Arabic YEH	U+064A	Farsi YEH	U+06CC	Medial / final ya

Table A.1: Arabic-to-Urdu character substitutions applied in Stage 0 normalisation.

In addition to these substitutions, Stage 0 also strips:

- Arabic diacritics: U+0610–U+061A and U+064B–U+065F.
- Superscript alef: U+0670.
- Tatweel / kashida: U+0640.
- Zero-width and directional control characters: U+200B–U+200F, U+202A–U+202E, U+2060–U+206F, U+FEFF.

-
- Urdu punctuation block: U+060C, U+060D, U+060E, U+060F, U+061B, U+061F, U+06D4 (Urdu full stop).
 - Extended Arabic punctuation: U+0600–U+060C, U+060E–U+061F.
 - All ASCII punctuation, symbols, letters, and digits.

Appendix B

REST API Specifications

This appendix records the full request and response schemas for the three pipeline endpoints. The implementation lives in `coral_app/backend/pipeline/pipeline_api.py`.

B.1 POST /align

Aligns each companion-model hypothesis to the named source-model hypothesis using weighted Levenshtein distance, then runs the secondary character-level alignment for split-merge classification.

```
1 {
2   "ensemble":      { "model_name": "raw transcript text", ... },
3   "source_model": "model_name"
4 }
```

Listing B.1: Request body schema for POST /align.

```
1 {
2   "source_model": "model_name",
3   "<model_name>": {
4     "normalized_attempt": ["w1", "w2", ...],
5     "aligned_attempt":   ["w1", "w2", ...],
6     "aligned_info":      ["MATCH", "SUBSTITUTION",
7                           ...],
8     "split_merge_aligned_attempt": ["chunk1", "chunk2", ...],
9     "split_merge_metadata": ["SAME", "SPLIT", "NOISE",
10                              ...],
11     "split_merge_metadata_base": ["SAME", "MERGE", ...],
12     "split_merge_aligned_info":  ["MATCH", "INSERTION", ...]
13   },
14 }
```

```

12     ... (one block per model) ...
13 }

```

Listing B.2: Response body schema for POST /align.

B.2 POST /oov

Detects OOV tokens using the dual model-disagreement + frequency criterion and ranks correction candidates by edit-distance + n-gram support.

```

1 {
2   "align_info": { ...alignment dict from /align... },
3   "freq_cutoff": 2000,      // optional, default 2000
4   "depth":       50,        // optional, BK-tree depth limit
5   "top_n":       10         // optional, candidates to return per
                             token
6 }

```

Listing B.3: Request body schema for POST /oov.

```

1 {
2   "oov_dict": ["oov_word_1", "oov_word_2", ...],
3   "metadata": {
4     "oov_word_1": {
5       "candidate_1": [edit_dist, tri_present, bi_present,
6                       uni_present, tri_freq, bi_freq, uni_freq],
7       "candidate_2": [...],
8       ...
9     },
10    ...
11  },
12  "columns": ["word", "Dist", "Trigram", "Bigram", "Unigram",
13             "Trifrequency", "Bifrequency", "Unifrequency"]
14 }

```

Listing B.4: Response body schema for POST /oov.

B.3 POST /correct

Applies the consensus voting algorithm to the alignment output, with OOV substitution overriding voting at flagged positions.

```

1 {
2   "align_info": { ...alignment dict from /align... },

```

```
3  "oov_metadata": { ...metadata dict from /oov...  }
4  }
```

Listing B.5: Request body schema for POST /correct.

```
1  {
2    "corrected": "final corrected Urdu transcript",
3    "source":    "source model normalised transcript",
4    "diff": [
5      { "pos": 3, "original": "x", "corrected": "y" },
6      ...
7    ]
8  }
```

Listing B.6: Response body schema for POST /correct.

B.4 Registry endpoints

The model registry is mounted at the /registry prefix and provides four endpoints (Section 4.1.4):

- GET /registry/models – list live models with seconds-since-last-ping.
- POST /registry/register – Kaggle notebook self-registration. Auth: X-Registry-Token.
- POST /registry/ping – 60-second heartbeat. Auth: X-Registry-Token.
- POST /registry/transcribe – multipart audio upload, fan-out to live models, parallel response collection. Auth: X-API-Token.

Appendix C

CORAL Source File Inventory

The CORAL source repository (C:\Users*\Desktop\CORAL-Urdu-ASR) contains:

- coral_app/backend/main_app.py – FastAPI entry point combining the pipeline and registry routers.
- coral_app/backend/pipeline/pipeline_api.py – the three pipeline endpoints.
- coral_app/backend/pipeline/coral_pipeline_functions.py
 - asr_aligner, build_oov_dict, extract_oov_metadata, apply_corrections.
- coral_app/backend/pipeline/coral_utility_functions.py – normalize_urdu, levenshtein, generate_alignment_data, is_oov, n-gram lookups.
- coral_app/backend/pipeline/bktree.py – BKTree and BKNode data structures.
- coral_app/backend/pipeline/coral_data_downloader.py – HuggingFace download + DuckDB bootstrap.
- coral_app/backend/model_registry/registry_api.py – model registry, eviction sweep, fan-out transcription.
- coral_app/frontend/my-next-app/app/ – Next.js front-end (ModeSelector.tsx, Pass0Speech.tsx, Pass1Input.tsx, Pass2Sieve.tsx, Pass3Correction.tsx, lib/api.ts).
- model_deploy_kaggle_notebooks/whisper_large_kaggle.ipynb, seamless_large_kaggle.ipynb – Kaggle GPU inference notebooks.

- `generate_ngrams.ipynb` – offline n-gram + BK-tree generation pipeline for the HuggingFace bucket.
- `urdu_asr_benchmark.ipynb`, `urdu_asr_pipeline.ipynb` – offline batch evaluation notebooks.
- `asr_results/`, `asr_ensemble/`, `asr_evaluation/` – per-model raw / ensemble / evaluation TSVs.
- `corpus/`, `vocab/`, `metadata/` – reference transcriptions, vocabulary, clip-duration metadata.
- `audios/` – raw audio sample files.
- `docs/` – architecture diagrams, the FYP-1 and FYP-2 final reports, and this report.

References

- [1] Anonymous. Better pseudo-labeling with multi-ASR fusion and error correction by SpeechLLM. *arXiv preprint arXiv:2506.11089*, 2025.
- [2] Rosana Ardila et al. Common voice: A massively-multilingual speech corpus. In *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC)*, pages 4211–4215, 2020.
- [3] Samee Arif, Aamir Junaid Khan, Mansoor Abbas, Agha Ali Raza, and Awais Athar. WER we stand: Benchmarking Urdu ASR models. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pages 5952–5961, 2025.
- [4] Walter A. Burkhard and Robert M. Keller. Some approaches to best-match file searching. *Communications of the ACM*, 16(4):230–236, 1973.
- [5] Wei Feng et al. Low-resource language adaptation with ensemble of PEFT approaches. In *Proceedings of APSIPA ASC*, 2024.
- [6] Jonathan G. Fiscus. A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (ROVER). In *Proceedings of the IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*, pages 347–354, 1997.
- [7] Igor Gitman et al. Confidence-based ensembles of end-to-end speech recognition models. In *Proceedings of Interspeech*, 2023.
- [8] Takaaki Hori et al. Delayed fusion: Integrating large language models into first-pass decoding in end-to-end speech recognition. *arXiv preprint arXiv:2501.09258*, 2025.
- [9] M. Koilakuntla et al. LLM-based ASR error correction for contact center domain-specific terminology. In *Proceedings of ICASSP*, 2024.
- [10] Rao Ma, Mengjie Qian, Mark Gales, and Kate Knill. ASR error correction using large language models. *arXiv preprint arXiv:2409.09554*, 2024.

- [11] C. Nagarathna et al. TruCLeS: True class lexical similarity score for novel confidence estimation in ASR. *IEEE Transactions on Audio, Speech, and Language Processing*, 2025.
- [12] S. A. Naqvi and M. A. Tahir. Code-mixed Urdu–English ASR for navigation domain using hybrid architecture. *Journal of Natural Language Processing*, 2024.
- [13] N. Parikh et al. Hybrid-E2E ASR ensemble with confidence calibration for low-resource irish language. In *Proceedings of LREC–COLING*, 2024.
- [14] A. Prakash et al. Multi-ASR fusion with LLM-based post-editing for high-quality pseudo-labels. In *Proceedings of Interspeech*, 2025.
- [15] Vineel Pratap et al. Scaling speech technology to 1,000+ languages. *arXiv preprint arXiv:2305.13516*, 2023.
- [16] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [17] Adithya Renduchintala, Shuoyang Ding, Matthew Wiesner, and Shinji Watanabe. Multi-modal data augmentation for end-to-end ASR. *arXiv preprint arXiv:1803.10299*, 2018.
- [18] Seamless Communication. SeamlessM4T: Massively multilingual and multimodal machine translation. *arXiv preprint arXiv:2308.11596*, 2023.